

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

А. А. Филиппов

ОПЕРАЦИОННЫЕ СИСТЕМЫ

Учебное пособие

Ульяновск
УлГТУ
2020

УДК 004.434 (075)
ББК 32.973.3я7
А??

Рецензенты:

Ведущий инженер-программист ФНПЦ АО «НПО «Марс», к. техн. наук, Радионова Ю. А.

Зав. кафедрой «Вычислительная техника» УлГТУ, к. техн. наук, доцент, Святков К. В.

Утверждено редакционно-издательским советом университета в качестве учебного пособия

Филиппов, Алексей Александрович

А?? Операционные системы: учебное пособие / А.А. Филиппов. — Ульяновск : УлГТУ, 2020.

ISBN ???-?-????-????-?

Учебное пособие по курсу «Операционные системы» содержит основные сведения, необходимые для понимания концепций, принципов, алгоритмов, структур и моделей, положенных в основу современных операционных систем. Особое внимание уделяется изучению абстракций операционных систем: процессы, потоки, виртуальная память, файл, ввод/вывод и т. д.

Для студентов вузов, обучающихся по направлениям «Программная инженерия», «Прикладная информатика (в экономике)» и другим, применяющим ЭВМ и современные операционные системы в своей деятельности.

**УДК 004.7 (075)
ББК 32.971.35я7**

ISBN ???-?-????-????-?

© Оформление. УлГТУ, 2020
© Филиппов А.А., 2020

ОГЛАВЛЕНИЕ

Введение	6
1 Введение в операционные системы	7
1.1 Операционная система как расширенная машина . . .	8
1.1.1 Системные вызовы	9
1.2 Операционная система как менеджер ресурсов	10
1.3 История развития операционных систем	12
1.4 Классы операционных систем	14
1.5 Структура ядра операционной системы	16
1.5.1 Монолитные системы	16
1.5.2 Микроядерные системы	17
1.6 Контрольные вопросы	18
2 Процессы и потоки	19
2.1 Модель процесса	20
2.2 Модель потока	24
2.3 Реализация многопоточности	27
2.4 Планирование процессов	28
2.4.1 Алгоритмы планирования пакетных систем . .	29
2.4.2 Алгоритмы планирования интерактивных си- стем	30
2.4.3 Алгоритмы планирования систем реального времени	33
2.5 Планирование потоков	33
2.6 Контрольные вопросы	34
3 Управление памятью	35
3.1 Абстракции для работы с оперативной памятью . . .	37
3.1.1 Прямой доступ к памяти	37
3.1.2 Выделение свободной памяти	39
3.1.3 Алгоритмы поиска свободного блока памяти .	41
3.1.4 Проблема нехватки физической памяти	41
3.1.5 Внутренняя и внешняя фрагментация	42
3.1.6 Адресное пространство	43
3.1.7 Свопинг	44
3.1.8 Базовый и ограничительный регистры	44
3.1.9 Страничная организация виртуальной памяти	47

3.1.10	Сегментная и сегментно-страничная организация виртуальной памяти	51
3.2	Алгоритмы замещения страниц	52
3.2.1	Оптимальный алгоритм	53
3.2.2	Алгоритм FIFO	53
3.2.3	Алгоритм «Второй шанс»	54
3.2.4	Алгоритм LRU	54
3.2.5	Алгоритм NRU	54
3.2.6	Алгоритм часы	55
3.2.7	Рабочий набор	55
3.2.8	Алгоритм WSClock	55
3.2.9	Алгоритм Page Fault Frequency	56
3.2.10	Пробуксовка	56
3.3	Контрольные вопросы	57
4	Файловые системы	58
4.1	Файлы	58
4.2	Каталоги	60
4.3	Адресация	62
4.4	Таблица разделов	63
4.5	Файловая система	64
4.5.1	Типы файловых систем	65
4.5.2	Методы расположения информации в файлах	67
4.5.3	Отслеживание свободных кластеров	70
4.5.4	Реализация каталогов	71
4.5.5	Непротиворечивость файловой системы	72
4.6	Контрольные вопросы	74
5	Ввод и вывод информации	75
5.1	Аппаратное обеспечение ввода-вывода	76
5.1.1	Контроллер устройства	76
5.1.2	Работа с контроллером	76
5.2	Прерывания	78
5.2.1	Вектор прерываний	79
5.2.2	Виды прерываний	80
5.2.3	Осуществление операций ввода-вывода	80
5.3	Программное обеспечение ввода-вывода	82

5.3.1	Задачи программного обеспечения ввода-вывода	83
5.3.2	Уровни программного обеспечения ввода-вывода	83
5.4	Контрольные вопросы	87
6	Состязательные и тупиковые ситуации	88
6.1	Состязательные ситуации	88
6.1.1	Механизмы решения проблемы возникнове- ния состязательных ситуаций	90
6.2	Тупиковые ситуации	92
6.2.1	Методы борьбы с взаимоблокировками	93
6.3	Контрольные вопросы	98
	Заключение	99
	Библиографический список	100

ВВЕДЕНИЕ

В процессе разработки любой программной системы возникает необходимость построения ее архитектуры в процессе разработки. Либо понимание архитектуры уже существующей программной системы необходимо для поддержания ее жизнеспособности. Многие современные подходы к разработке сложных промышленных программных систем требуют от разработчика умения оперировать абстракциями различного уровня.

В данном учебном пособии содержится материал, используя который можно приступить к изучению базовых абстракций, положенных в основу современных операционных систем:

1. Процессы и потоки.
2. Виртуальная память.
3. Файлы и каталоги.
4. Ввод/вывод.
5. Системные вызовы.

Конечно, материал учебного пособия не раскрывает все тонкости архитектуры современных операционных систем. В частности, не рассмотрены детально операционные системы семейства Windows NT, GNU/Linux, BSD и т. д. Для получения дополнительной информации следует обращаться к литературе из предложенного списка.

А.А. Филиппов

1 ВВЕДЕНИЕ В ОПЕРАЦИОННЫЕ СИСТЕМЫ

В различных источниках очень сложно найти устоявшееся определение понятия операционная система. Исследователи и специалисты в данной области рассматривают операционную систему с различных позиций:

- как библиотеку функций, которые можно использовать при разработке прикладных программ,
- в виде набора различных абстракций над аппаратным обеспечением вычислительной системы,
- как средство для организации взаимодействия между пользователем и программным обеспечением,
- как систему, распределяющую и управляющую ресурсами вычислительной системы и т. д.

На рисунке 1.1 обозначается место операционной системы в вычислительной системе – между программным (прикладным) и аппаратным обеспечением.

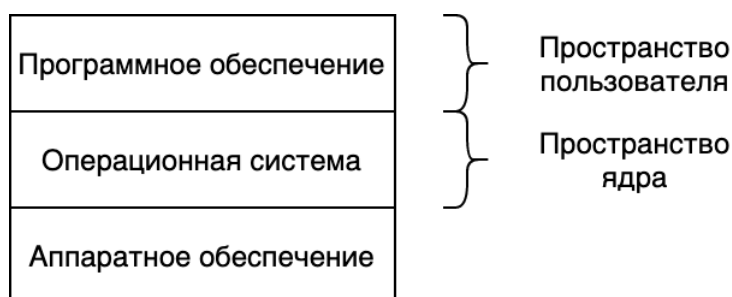


Рис. 1.1. Место операционной системы в вычислительной системе

Как видно из рисунка 1.1 операционная система выполняется в пространстве ядра, а прикладное программное обеспечение – в пространстве пользователя. Данные пространства реализуются на уровне режимов работы центрального процессора.

Под пространством ядра обычно понимается такая область памяти, в которой выполняется ядро операционной системы (память ядра). Программа, работающая в пространстве ядра имеет полный доступ к оперативной памяти и устройствам ввода-вывода. Прикладное программное обеспечение обычно выполняется в пространстве пользователя и имеет ограниченные возможности по работе с памятью и устройствами ввода-вывода. Для взаимодействия с устройствами ввода-вывода из пространства пользователя применяются системные вызовы.

В рамках данного пособия операционная система будет рассмотрена с двух позиций [1]:

1. Операционная система как расширенная машина.
2. Операционная система как менеджер ресурсов.

1.1 Операционная система как расширенная машина

Операционная система как расширенная машина предоставляет программному обеспечению набор абстракций для работы с составными частями вычислительной системы: центральный процессор, оперативная память, постоянная память, различные устройства ввода-вывода.

В данном случае под абстракцией понимается некоторая «надстройка» над сложной сущностью, которая позволяет скрыть сложность и облегчить работу с такой сущностью. Например, абстракцией является водопровод, электрическая сеть, локальная вычислительная сеть и т. д. Также различные абстракции активно используются при разработке программного обеспечения. Самым главным условием при разработке абстракций является максимальная простота полученного решения – абстракция не должна быть сложнее или равной по сложности той сущности, над которой она формируется.

К абстракциям, предоставляемым операционной системой в рамках функции операционная система как расширенная машина, можно отнести процессы и потоки (см. главу 2), виртуальную память (см. главу 3), файловые системы (см. главу 4), ввод-вывод данных (см. главу 5) и системные вызовы.

Для лучшего понимания функции операционная система как расширенная машина обратимся к рисунку 1.2.

Как видно из рисунка 1.2, вычислительная система содержит в своем составе следующие устройства: HDD1, HDD2, SSD, Printer. Для работы с аппаратным обеспечением программное обеспечение обращается к операционной системе. Организация взаимодействия между операционной системой и устройствами осуществляется с помощью соответствующих драйверов: Driver1, Driver2, Driver3 и Driver4. При этом для вызова функций драйверов Driver1, Driver2 и Driver3 операционная система использует единый набор команд.

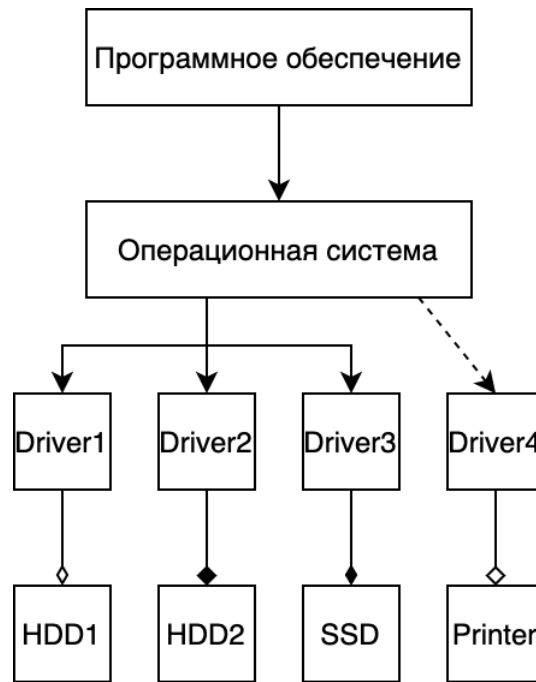


Рис. 1.2. Операционная система как расширенная машина

Драйверы используются для связи высокоуровневого программного интерфейса, реализованного в рамках операционной системы для работы с устройствами одного типа: устройства хранения, сетевые устройства, устройства печати и т. д., и низкоуровневыми интерфейсами контроллера устройств ввода-вывода. Фактически, высокоуровневую часть драйвера устройства можно рассматривать в качестве интерфейса в рамках парадигмы объектно-ориентированного программирования, а остальную часть драйвера – в качестве реализации такого интерфейса. Более подробно работа с устройствами ввода-вывода представлена в главе 5.

Таким образом, при смене оборудования разработчику прикладного программного обеспечения не требуется вносить изменения в программный код, разбираться в тонкостях работы конкретного устройства ввода-вывода и принципах работы с его контроллером. Вместо этого, для взаимодействия с устройствами различного типа используется единообразный набор методов.

1.1.1 Системные вызовы

Системный вызов (System call) – обращение прикладной программы к ядру операционной системы для выполнения какой-либо операции.

Так как прикладное программное обеспечение выполняется в пространстве пользователя, для выполнения межпроцессных операций или операций, требующих доступа к оборудованию, необходимо обратиться к ядру операционной системы. Ядро операционной системы выполняется в пространстве ядра и имеет полный доступ к оперативной памяти и устройствам ввода-вывода. В зависимости от полномочий запущенного экземпляра программы, которая производит обращение к системному вызову, ядро исполняет данный вызов либо отказывает в исполнении.

С точки зрения программиста, системный вызов обычно выглядит как обращение к методам системной библиотеки, но требует наличия определенных привилегий и приводит к переключению контекста исполнения запущенного экземпляра программы. Контекст исполнения и процесс смены контекста исполнения рассмотрены в главе 2.

Каждый системный вызов имеет уникальный идентификатор. Для обращения к конкретному системному вызову необходимо поместить номер этого системного вызова в определенный регистр центрального процессора. Аргументы для выполнения системного вызова можно передавать через регистры центрального процессора либо передавать указатель на некоторую область памяти, которая содержит необходимые аргументы, или указатель на стек. Указатель позволяет передавать в качестве аргумента различные типы и объемы данных, а не ограничиваться размером и количеством регистров центрального процессора. Однако, передача указателя является более дорогостоящей операцией по сравнению с использованием регистров.

1.2 Операционная система как менеджер ресурсов

Операционная система как менеджер ресурсов управляет распределением ресурсов вычислительной системы между множеством запущенных экземпляров программного обеспечения и делает это прозрачно для пользователя и разработчика.

Операционная система распределяет ресурсы вычислительной системы во времени и в пространстве.

К временным ресурсам относятся ресурсы центрального процессора – процессорное время. Фактически, операционная систе-

ма определяет порядок (очередь) выполнения запущенных экземпляров программного обеспечения для их выполнения на вычислительном ядре центрального процессора. Временной характер распределения ресурсов центрального процессора (вычислительного ядра) связан с особенностью работы тракта данных.

Тракт данных – это часть вычислительного ядра центрального процессора, состоящая из арифметико-логического устройства, а также его входов и выходов, связанных с регистрами центрального процессора через шины данных.

Функционирование арифметико-логического устройства тракта данных зависит от линий управления, количество которых может варьироваться в зависимости от архитектуры центрального процессора. Линии управления служат для определения типа операции, для разрешения входных сигналов (определения источника входных данных), для инверсии значения входного сигнала, для переноса бита в младший разряд (увеличение на единицу). В зависимости от комбинации сигналов линий управления тракта данных определяется источник и приемник данных, выполняемое действие арифметико-логического устройства: сложение, сдвиг, сравнение и т. д.

Для предотвращения искажения данных в случае, если один и тот же регистр считывается и записывается в рамках одного цикла, процессы чтения и записи происходят в разных частях цикла. Когда в качестве источника входных данных для арифметико-логического устройства выбирается один из регистров, его значение помещается на входную шину данных в начале цикла и хранится там на протяжении всего цикла. Затем результат работы арифметико-логического устройства поступает на выходную шину данных. Перед окончанием цикла работы содержание выходной шины передается в один или несколько регистров, например, в тот же регистр, с которого сигнал поступил на входную шину. Точная синхронизация тракта данных делает возможным считывание и запись одного и того же регистра за один цикл.

Таким образом, в одну единицу времени на одном вычислительном ядре возможно выполнение только одного запущенного экземпляра программного обеспечения.

К пространственным ресурсам можно отнести ресурсы оперативной памяти и ресурсы устройств хранения данных. В данном

случае операционной системе необходимо выделить некоторый диапазон (диапазоны) адресов оперативной памяти или устройства хранения данных каждому запущенному экземпляру программного обеспечения и обеспечить необходимый уровень безопасности доступа к таким ресурсам.

1.3 История развития операционных систем

Для лучшего понимания роли операционной системы в истории развития информационных технологий рассмотрим основные этапы развития операционных систем. Этапы развития операционных систем тесно связаны с поколениями электронно-вычислительных машин (ЭВМ).

В первом поколении ЭВМ (1945-1960) в качестве интерфейса взаимодействия с пользователем применялись коммутационные панели, операционных систем тогда не существовало.

Первые операционные системы появились с появлением **второго поколения** ЭВМ (1955-1970). Во втором поколении ЭВМ в качестве интерфейса взаимодействия с пользователем для ввода информации обычно применялись перфокарты, а для вывода информации магнитные ленты.

В то время программное обеспечение представляло собой множество перфокарт, часть из которых содержала исходный код самой программы, а часть входные данные. После загрузки информации с перфокарт производился процесс компиляции программы и ее дальнейший запуск. Полученные в результате работы программы данные записывались на магнитную ленту, которую затем переносили на другую более дешевую ЭВМ и производили распечатку результатов.

Для работы с ЭВМ программисту необходимо было записаться на определенное время. Если в процессе компиляции или выполнения программы возникали ошибки, время тратилось впустую. Вычислительное время ЭВМ стоило дороже рабочего времени человека и для эффективного использования времени ЭВМ были разработаны специальные программы – мониторы, которые автоматизировали работу оператора, выполняя следующие функции:

- загрузка и обработка перфокарт, содержащих программу – задание,
- ожидание окончания программы и переход к следующему заданию,
- если срок выполнения какого-либо задания превышал некоторый максимальный порог, то такое задание прерывалось и происходил переход к следующему заданию.

В процессе работы программа-монитор постоянно располагалась в оперативной памяти вычислительной системы. Обращение к памяти происходило напрямую, без использования абстракций и механизмов защиты. Таким образом, существовала вероятность, что в результате выполнения задания текущая программа могла перезаписать часть памяти монитора. При окончании текущего задания и запуске монитора изменение его образа памяти приводило к возникновению ошибки из-за которой монитор завершал свою работу. Для решения данной проблемы диапазон памяти, в которой был запущен монитор, был защищен от записи со стороны других программ. Если какая-либо программа пыталась обратиться к памяти монитора, возникала соответствующая ошибка. Со временем данная функция была реализована аппаратно в центральном процессоре, что явилось прообразом пространства ядра.

ЭВМ второго поколения можно отнести к системам пакетной обработки данных, не предполагающих активное взаимодействие с пользователями.

Со временем ЭВМ развивались, становились более производительными, компактными и дешевыми. Программы-мониторы также развивались и усложнялись, в некоторый момент времени такие программы стали называть операционными системами.

В третьем поколении ЭВМ (1965-1980) вычислительное время ЭВМ стало стоить дешевле рабочего времени человека и появилась необходимость реализации многопользовательского доступа к ресурсам ЭВМ. Стали появляться компьютерные терминалы, которые позволяли пользователям взаимодействовать с ЭВМ. Возникла необходимость одновременного запуска нескольких программ в рамках одной ЭВМ с возможностью их параллельного исполнения. Таким образом, ЭВМ третьего поколения можно отнести к интерактивным системам, предполагающим вза-

имодействие с пользователями, а операционные системы третьего поколения содержали функции управления ресурсами ЭВМ и поддерживали многозадачность, работу с виртуальной памятью и файловыми системами.

Также в операционных системах третьего поколения появилась возможность запуска программного обеспечения в рамках семейства ЭВМ. До этого момента, при выпуске новой ЭВМ с новой архитектурой было необходимо практически полностью переписать как операционную систему, так и программное обеспечение, что требовало больших временных и финансовых затрат. Третье поколение операционных систем позволило запускать программное обеспечение, написанное для ЭВМ одного семейства, без необходимости адаптации программ к особенностям архитектуры конкретной ЭВМ.

С приходом **четвергового поколения** ЭВМ (с 1975) компьютеры стали доступны для покупки широкому кругу покупателей, появилась необходимость в разработке интерфейсов, ориентированных на неподготовленных пользователей. Также в данное время происходит рост и развитие локальных сетей. Во многие операционные системы были добавлены сетевые функции. В настоящий момент существуют как универсальные операционные системы, способные работать на широком спектре оборудования и решать широкий класс задач, так и специализированные операционные системы, выполняющие конкретные функции в условиях ограниченных вычислительных ресурсов, например, в бытовых приборах, сетевом оборудовании, автомобилях и т. д.

1.4 Классы операционных систем

Обычно выделяют следующие классы операционных систем:

1. Операционные системы мейнфреймов – ориентированы на выполнение пакетных задач. Эффективность операционных систем мейнфреймов оценивается в количестве выполненных задач в единицу времени.
2. Серверные операционные системы – ориентированы на обработку большого числа запросов со стороны клиентов. Эффективность серверных операционных систем оценивается

в количестве обработанных запросов в единицу времени и среднем времени ожидания ответа от сервера.

3. Многопроцессорные операционные системы – ориентированы на работу в вычислительных системах с несколькими центральными процессорами и/или вычислительными ядрами. Эффективность многопроцессорных операционных систем оценивается в оптимальном распределении задач по вычислительным ядрам и/или центральным процессорам.
4. Операционные системы персональных компьютеров – ориентированы на взаимодействие с пользователями (работа в интерактивном режиме). Эффективность операционных систем персональных компьютеров оценивается в среднем времени отклика системы на запросы пользователя.
5. Операционные системы карманных персональных компьютеров – ориентированы на взаимодействие с пользователями, работу в режиме ограниченных вычислительных ресурсов и низкого энергопотребления. Эффективность операционных систем карманных персональных компьютеров оценивается в среднем времени отклика системы на запросы пользователя и времени автономной работы.
6. Встроенные операционные системы – имеют ограниченный набор возможностей и функций, ориентированы на работу в специализированных вычислительных системах с ограниченными вычислительными ресурсами.
7. Операционные системы реального времени – ориентированы на обработку данных с привязкой ко времени выполнения физических процессов (жестко ограниченный период времени), например, в составе системы управления ядерным реактором, системы автопилота и т. д.

В настоящий момент данная классификация может считаться достаточно условной, так как существуют современные универсальные операционные системы (Windows NT, GNU/Linux), которые можно отнести практически ко всем представленным классам.

1.5 Структура ядра операционной системы

Обычно операционную систему рассматривают в виде совокупности некоторых компонентов из которых выделяют ядро операционной системы. Ядро операционной системы выполняется в пространстве ядра и реализует основные функции операционной системы.

В настоящий момент существует несколько подходов к формированию архитектуры ядра операционной системы:

- монолитные системы,
- многоуровневые системы,
- микроядерные системы,
- клиент-серверные системы,
- виртуальные машины,
- экзоядра.

Рассмотрим более подробно некоторые из подходов.

1.5.1 Монолитные системы

В монолитных системах ядро представляет собой одну большую программу, состоящую из множества методов. Пример монолитной системы представлен на рисунке 1.3.

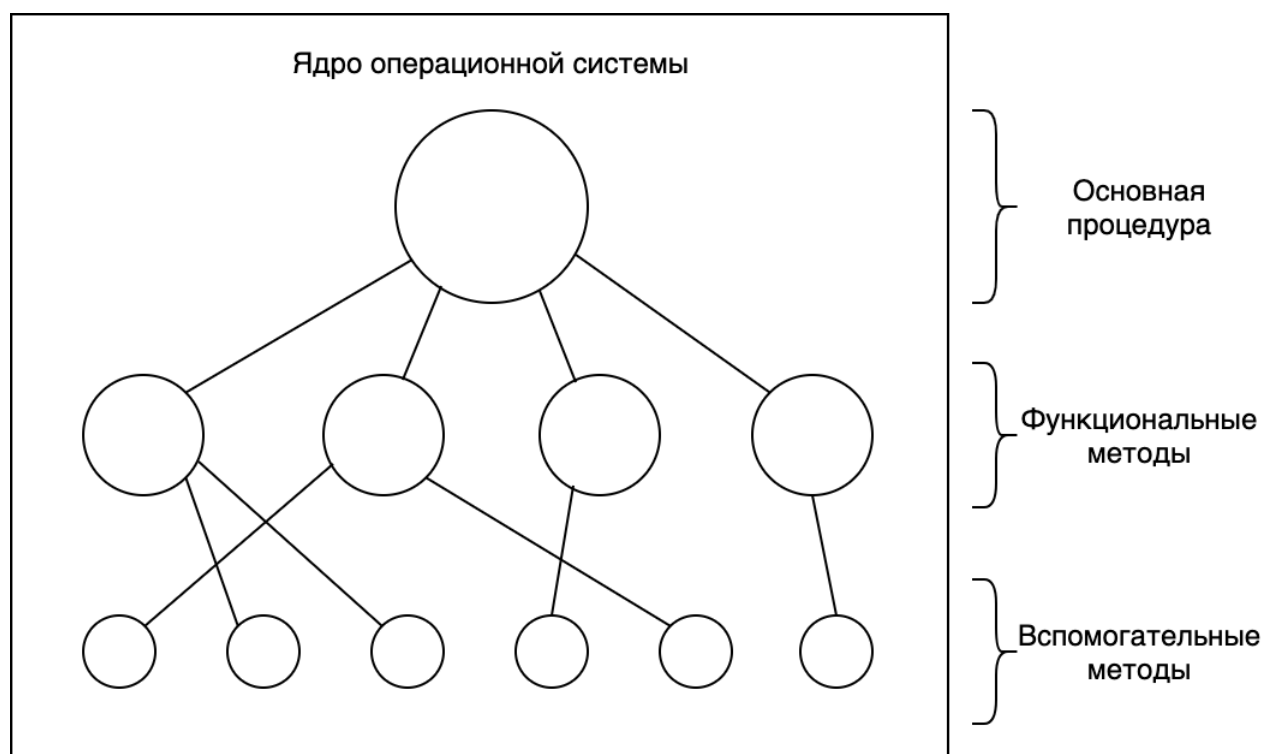


Рис. 1.3. Иллюстративный пример монолитной системы

В качестве преимуществ монолитной системы можно выделить:

- высокую скорость работы за счет нахождения всего ядра в одной области памяти (одна программа),
- удобство отладки и разработки.

К недостаткам монолитной системы можно отнести низкую отказоустойчивость. Так как ядро представляет собой программу, выполняющуюся в одной области памяти, возникновение любой ошибки может привести к остановке ядра и невозможности работы вычислительной системы.

1.5.2 Микроядерные системы

В отличие от монолитной системы, микроядерная система представляет собой несколько взаимодействующих программ. При этом ядро операционной системы, выполняющееся в пространстве ядра, имеет минимальный размер программного кода и реализует минимально необходимые функциональные возможности: обработка прерываний, таймеры, обмен сообщениями. Остальная часть операционной системы выполняется в пространстве пользователя. Для взаимодействия между компонентами микроядерной системы используется механизм обмена сообщениями, посредником в котором выступает микроядро. Фактически, взаимодействие компонентов микроядерной системы похоже на общение через сервис обмена сообщениями: компоненты в пространстве ядра являются клиентами, а микроядро является сервером. Пример микроядерной системы представлен на рисунке 1.4.

К преимуществам микроядерной системы можно отнести высокую отказоустойчивость за счет выполнения большей части компонентов операционной системы в пространстве пользователя. Если в каком-либо компоненте пространства пользователя микроядерной системы происходит сбой, то это не приведет к остановке всей системы, и такой компонент можно перезапустить.

К недостаткам микроядерной системы можно отнести:

- низкую скорость работы за счет необходимости обмена сообщениями между компонентами (совокупность нескольких программ),

- усложнение процессов отладки и разработки.

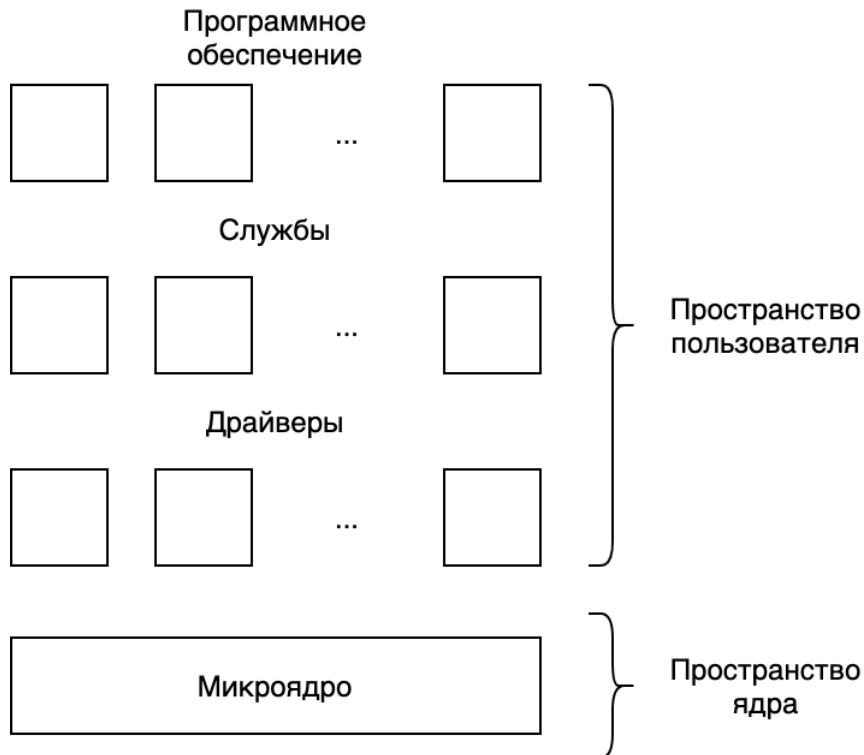


Рис. 1.4. Иллюстративный пример микроядерной системы

1.6 Контрольные вопросы

1. Перечислите основные функции операционной системы.
2. Перечислите основные абстракции операционной системы.
3. Расскажите про монолитный подход к построению ядра операционной системы.
4. Расскажите про микроядерный подход к построению ядра операционной системы.
5. Что такое системный вызов?
6. Для чего используются системные вызовы?
7. Каким образом происходит передача параметров при обращении к системному вызову?
8. В чем различия между пространством ядра и пространством пользователя?

2 ПРОЦЕССЫ И ПОТОКИ

Процесс является базовой абстракцией любой операционной системы, так как позволяет представить запущенный экземпляр программы. В данном случае программа (исполняемый файл) рассматривается как набор инструкций для выполнения, а процесс как сущность – процесс выполнения данной программы. У одной программы может быть несколько запущенных экземпляров, для каждого из которых будет создан отдельный процесс.

Как было рассмотрено в главе 1, процесс представляет собой абстракцию для управления ресурсами центрального процессора – процессорным временем. На одном вычислительном ядре центрального процессора в одну единицу времени может выполняться только один процесс. Следовательно, в операционной системе должны содержаться механизмы для формирования очереди выполнения процессов – подсистема планирования выполнения.

Подсистема планирования выполнения должна упорядочить процессы таким образом, чтобы ресурсы центрального процессора были использованы эффективно. Критерии эффективности планирования зависят от класса операционной системы и типа решаемых задач.

Например, в интерактивных системах, ориентированных на взаимодействие с пользователем, необходимо выделять каждому процессу некоторый квант процессорного времени для поддержания иллюзии многозадачности. Например, вычислительная система содержит в своем составе один центральный процессор с одним вычислительным ядром, а операционная система, запущенная на такой вычислительной системе, относится к операционным системам персональных компьютеров. Допустим, что в системе выполняется 3 процесса, с которыми взаимодействует пользователь. Очевидно, что в одну единицу времени может выполняться только один процесс, но пользователь может использовать один процесс в фоновом режиме, а между двумя оставшимися процессами активно переключаться.

Таким образом, подсистема планирования выполнения должна выделять квант времени всем процессам поочередно – начиная с первого процесса и перехода к следующему. В том случае, если достигнут конец списка, необходимо вновь перейти к первому процессу. Так как квант времени является достаточно

коротким, например, 20 миллисекунд, пользователю кажется, что процессы выполняются параллельно, несмотря на наличие в системе лишь одного вычислительного ядра.

На рисунке 2.1 представлена иллюстрация рассмотренного выше примера планирования процессов.

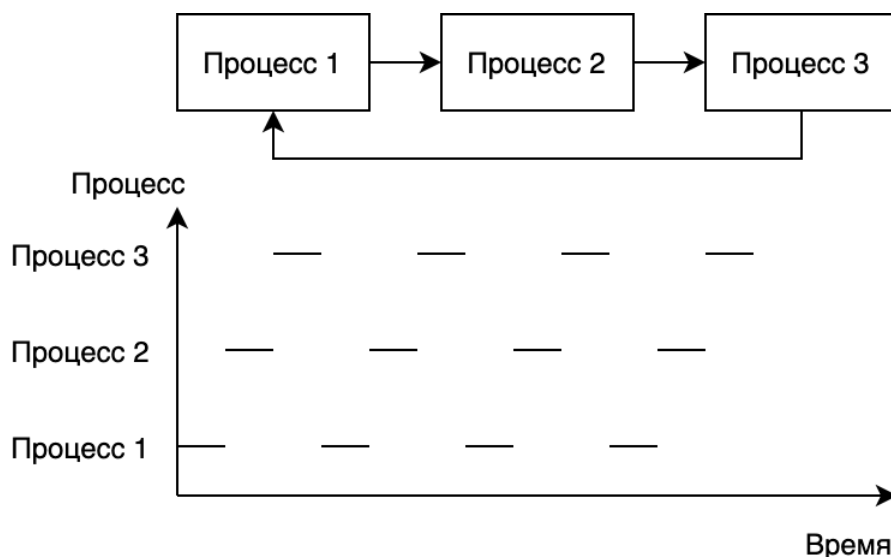


Рис. 2.1. Иллюстративный пример планирования процессов

Как видно из рисунка 2.1, процессы выполняются последовательно, но из-за короткого кванта времени создается иллюзия параллельного выполнения процессов (см. график).

2.1 Модель процесса

Для лучшего понимания абстракции «процесс» необходимо рассмотреть варианты возможных состояний, в которых может находиться процесс в большинстве разновидностей операционных систем (рис. 2.2).

Как видно из рисунка 2.2, процесс может находиться в пяти состояниях:

1. Запуск.
2. Готовность.
3. Выполнение.
4. Блокировка.
5. Завершение.

Запуск процесса может быть произведен при:

1. Инициализации операционной системы.

2. Выполнении работающим процессом системного вызова, предназначенного для создания процесса.
3. Запросе пользователя на создание нового процесса.
4. Инициализации пакетного задания.

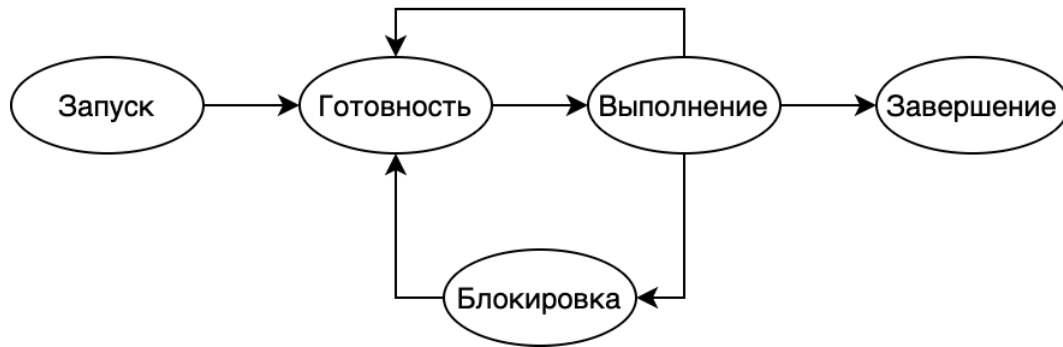


Рис. 2.2. Состояния процесса

При запуске процесса необходимо создать запись в таблице процессов и выделить для процесса виртуальное адресное пространство. Более подробно работа с памятью рассмотрена в главе 3. Таблица процессов используется в операционной системе для хранения информации о запущенных процессах. Каждая запись таблицы процессов имеет уникальный идентификатор – идентификатор процесса (Process ID, PID) и обычно содержит следующий набор полей:

1. Значения регистров центрального процессора.
2. Значение счетчика команд (специальный регистр).
3. Слово состояния программы.
4. Указатель на стек.
5. Состояние процесса.
6. Приоритет.
7. Параметры планирования.
8. Родительский процесс.
9. Группа процесса.
10. Сигналы.
11. Время запуска процесса.
12. Использованное время процессора.
13. Время процессора, использованное дочерними процессами.
14. Время следующего аварийного сигнала и другие в зависимости от реализации.

Схематично виртуальное адресное пространство процесса представлено на рисунке 2.3.

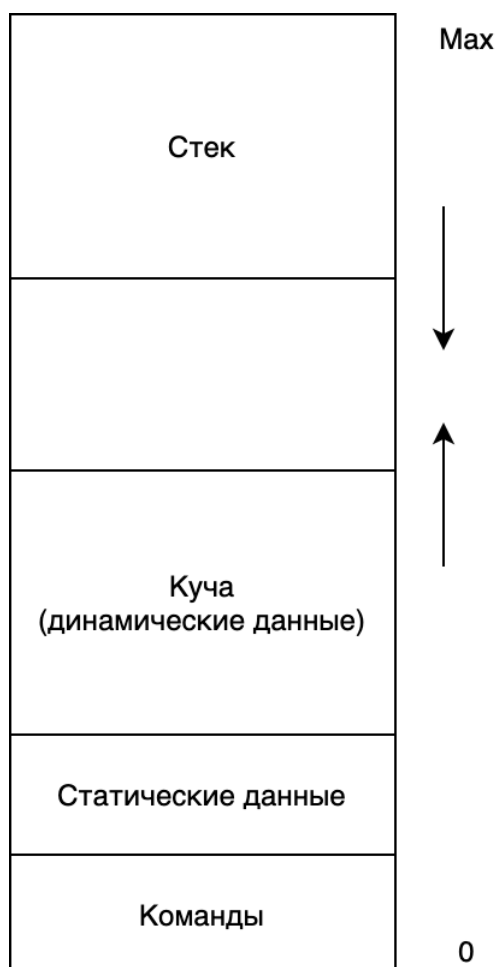


Рис. 2.3. Иллюстративный пример виртуального адресного пространства

Как видно из рисунка 2.3, виртуальное адресное пространство процесса содержит следующие сегменты:

- сегмент для хранения команд – содержит инструкции для выполнения, описанные в исполняемом файле программы;
- сегмент для хранения статических данных – содержит проинициализированные переменные с простым типом данных;
- куча (heap) – содержит динамические типы данных, может увеличивать свой размер в процессе выполнения программы, приближаясь к старшим адресам. Если размер кучи достигает определенного максимума, возникает ошибка переполнения кучи;
- стек (stack) – содержит информацию для восстановления потока управления при возврате из подпрограмм (процедур,

функций) и/или для возврата в программу из обработчика прерываний, может увеличивать свой размер в процессе выполнения программы, приближаясь к младшим адресам. Если размер стека достигает определенного максимума, возникает ошибка переполнения стека.

Максимальный размер адресного пространства зависит от разрядности центрального процессора (32 бита или 64 бита), разрядности операционной системы и деталей реализации адресного пространства в операционной системе.

После запуска процесс помещается в очередь процессов для планирования, когда очередь выполнения доходит до текущего процесса, процесс переходит в состояние выполнения. В состоянии выполнения процессу выделяется квант времени, в рамках которого процесс использует вычислительное ядро для выполнения команд.

После окончания кванта времени процесс прерывается, происходит сохранение его контекста исполнения в соответствующую запись таблицы процессов. Для поиска нужной записи таблицы процессов используется идентификатор процесса. После чего процесс переходит в состояние готовности и ожидает выполнения.

Под контекстом исполнения в данном случае подразумевается совокупность значений регистров центрального процессора и значение счетчика команд. Контекст исполнения сохраняется в записи таблицы процессов для сохранения состояния выполнения прерванного процесса. Далее происходит выбор следующего для выполнения процесса и происходит восстановление его контекста исполнения – загрузка значений регистров и счетчика команд из таблицы процессов в соответствующие регистры центрального процессора. После чего процесс выполняется.

Из состояния выполнения процесс может быть переведен в состояние блокировки. Обычно такая необходимость возникает при выполнении обращения к устройству ввода-вывода, скорость работы с которым намного меньше скорости работы тракта данных центрального процессора. При переходе в состояние блокировки процесс также прерывается, а его контекст исполнения сохраняется в записи таблицы процессов. Процесс находится в состоянии блокировки до тех пор, пока устройство ввода-вывода не завершит выполнение необходимых операций: запись или чтение

данных. После чего процесс переводится в состояние готовности и ожидает выполнения. Более подробно работа с устройствами ввода-вывода рассмотрена в главе 5.

Состояние блокировки позволяет снизить влияние операций ввода-вывода на производительность вычислительной системы. Если процесс не будет переведен в состояние блокировки при обращении к устройствам ввода-вывода, то оставшийся квант времени будет потрачен впустую.

Процесс может быть завершен:

1. Добровольно (обычное завершение).
2. При возникновении ошибки (добровольное завершение).
3. При возникновении фатальной ошибки (принудительное завершение).
4. Завершение текущего процесса другим процессом (принудительное завершение).

2.2 Модель потока

Поток выполнения (поток, нить, thread) – наименьшая единица обработки команд, выполнение которой может быть спланировано подсистемой планирования выполнения.

Обычно каждый процесс содержит минимум один поток выполнения, в рамках которого выполняются команды данного процесса. Некоторые операционные системы позволяют создавать в рамках одного процесса несколько потоков выполнения. Все потоки работают в едином адресном пространстве процесса и имеют полный доступ к памяти и ресурсам процесса. При наличии нескольких вычислительных ядер и/или процессоров потоки могут выполняться параллельно.

Использование нескольких потоков в одном процессе позволяет значительно повысить производительность приложений на вычислительных системах с несколькими вычислительными ядрами и/или процессорами за счет параллельного выполнения нескольких задач на разных вычислительных ядрах и/или процессорах, а также повысить отзывчивость приложения при наличии одного вычислительного ядра за счет запуска «тяжелых» задач в отдельном потоке выполнения. Например, если выполнять некоторые расчеты в основном потоке выполнения, то интерфейс приложе-

ния будет недоступен до завершения такой операции. Данную проблему можно решить путем выполнения расчетов в отдельном потоке.

Возможность создания и выполнения нескольких потоков в рамках одного процесса называется многопоточностью. В том случае, если операционная система поддерживает многопоточность, в ядре операционной системы содержится таблица потоков. Каждая запись таблицы потоков соответствует некоторому потоку выполнения, имеет уникальный идентификатор – идентификатор потока (thread id, TID), и содержит следующие поля:

1. Значения регистров центрального процессора.
2. Значение счетчика команд (специальный регистр).
3. Слово состояния.
4. Указатель на стек и другие в зависимости от реализации.

Таким образом, при поддержке многопоточности поток становится единицей выполнения и планирования – у каждого потока есть свой контекст исполнения, а процесс становится контейнером, содержащим ссылки на ресурсы и память, необходимые для выполнения потоков. Запись таблицы процессов содержит следующие поля:

1. Адресное пространство.
2. Глобальные переменные.
3. Открытые файлы.
4. Дочерние процессы.
5. Необработанные аварийные сигналы.
6. Сигналы и обработчики сигналов.
7. Учетная информация и другие в зависимости от реализации.

Для лучшего понимания различий между процессами и потоками рассмотрим следующий пример – существует необходимость разработки программы, способной обрабатывать поступающие от пользователей запросы. После обработки запроса необходимо вернуть результат пользователю. В одну единицу времени может поступать несколько запросов, требуется выполнять такие запросы сразу, без использования очереди запросов.

В первом случае реализовать подобную систему можно с использованием нескольких процессов: процесс, который принимает пользовательские запросы, для обработки каждого запроса

создается отдельный процесс. Рассмотрим преимущества и недостатки данного подхода.

К преимуществам можно отнести:

- работоспособность подхода практически на всех операционных системах,
- высокая отказоустойчивость системы.

К недостаткам:

- создание отдельных процессов для обработки запросов пользователя является дорогостоящей операцией – для создания процесса необходимо выполнить системный вызов, для каждого процесса необходимо создать виртуальное адресное пространство (см. главу 3);
- разработка механизма передачи данных между процессами для возврата результата обработки запроса пользователю – каждый процесс работает в своем собственном виртуальном адресном пространстве, и доступ к адресному пространству другого процесса возможен через специальные средства операционной системы либо собственные механизмы, основанные, например, на сетевом взаимодействии, общих файлах и т. д. Защита адресных пространств от доступа со стороны других процессов обеспечивает безопасную работу нескольких процессов с оперативной памятью (см. главу 3). Использование механизмов межпроцессорного взаимодействия также снижает скорость работы системы.

Во втором случае, подобную задачу можно решить с применением многопоточности. В таком случае, все запросы со стороны пользователя будут обработаны в рамках отдельного потока одного процесса.

Преимуществами данного подхода являются:

- низкие накладные расходы на создание потоков – нет необходимости в создании адресного пространства, в некоторых случаях для создания потока не требуется обращение к ядру операционной системы через системный вызов;
- все потоки работают в одном адресном пространстве и используют общую память – нет необходимости в создании/использовании механизмов межпроцессорного взаимодействия.

Также можно выделить следующие недостатки:

- данный подход к организации системы является менее надежным по сравнению с системой, основанной на нескольких процессах, так как сбой в потоке исполнения может привести к останову всей системы,
- требуется наличие поддержки многопоточности.

2.3 Реализация многопоточности

Для использования многопоточности необходима поддержка со стороны операционной системы либо со стороны среды исполнения программ, реализованной в виде библиотеки или виртуальной машины (среды исполнения).

При поддержке многопоточности со стороны операционной системы потоки выполнения являются потоками выполнения ядра (модель 1:1). Если же поддержка многопоточности реализована средствами среды исполнения программ, то говорят о потоках выполнения в пространстве пользователя (модель N:1). Возможен также и гибридный подход (модель M:N).

На рисунке 2.4 представлены варианты реализации поддержки многопоточности: в пространстве ядра и в пространстве пользователя.

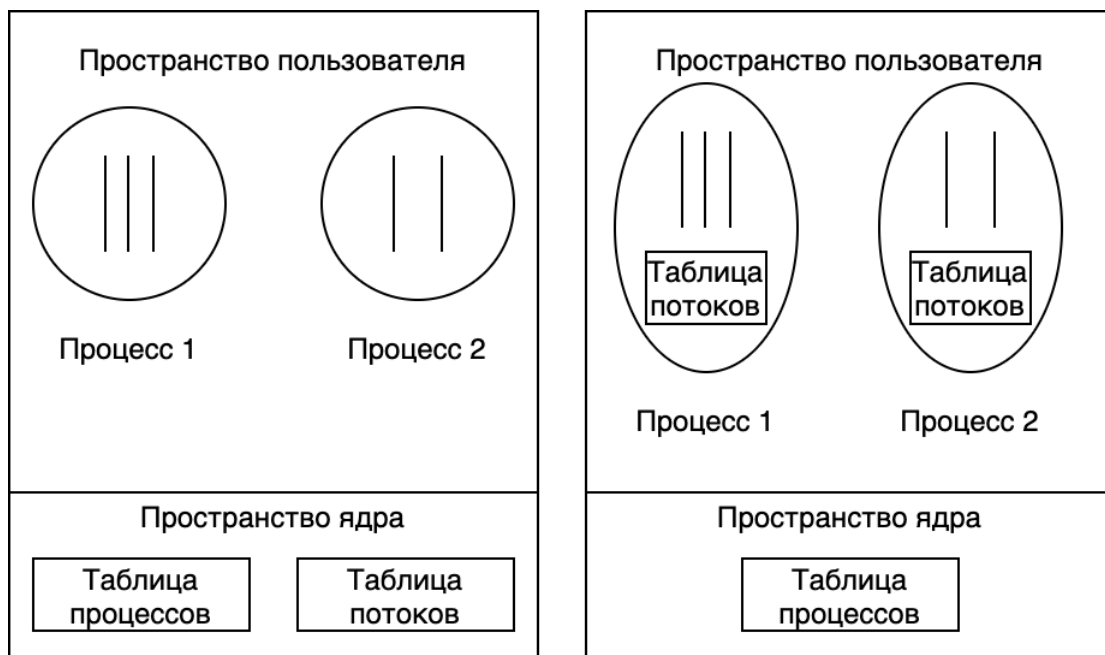


Рис. 2.4. Варианты реализации поддержки многопоточности

Как видно из рисунка 2.4, для поддержки многопоточности в пространстве ядра необходимо добавить данную функциональность в ядро операционной системы. В таком случае ядро операционной системы будет иметь полную информацию о потоках и учитывать ее при планировании. Например, в случае обращения потока к устройству ввода-вывода в состоянии блокировки будет переведен только этот поток. Однако, в данном случае, для создания потока необходимо выполнение системного вызова, что обычно приводит к переключению контекста исполнения и дополнительным накладным расходам.

В случае, если операционная система не поддерживает многопоточность, данную функциональность можно реализовать в пространстве пользователя с применением среды исполнения программ. В данном случае операционная система будет планировать только процессы, а созданием и планированием дополнительных потоков будет заниматься среда исполнения программ – таблица потоков располагается в пространстве пользователя и привязана к конкретному процессу. Так как таблица потоков расположена в пространстве пользователя, создание потока не будет требовать переключения контекста исполнения и дополнительных накладных расходов, но при обращении некоторого потока пространства пользователя к устройству ввода-вывода в состоянии блокировки будет переведен весь процесс. Также реализация потоков в пространстве пользователя не позволяет получить выигрыш в вычислительных системах с несколькими вычислительными ядрами и/или процессорами.

При использовании гибридного подхода некоторое количество потоков пространства пользователя отображаются на некоторое количество потоков выполнения ядра. Гибридный подход сложен в реализации – требует координации между планировщиком потоков среды исполнения программ и подсистемой планирования выполнения операционной системы, но при этом является более гибким решением.

2.4 Планирование процессов

При планировании процессов обычно выделяет следующие типы задач:

- задачи, ограниченные скоростью вычислений – основная часть процессорного времени тратится на вычисления;
- задачи, ограниченные скоростью работы устройств ввода-вывода – основная часть процессорного времени тратится на ожидание завершения операций ввода-вывода. Процессор простаивает, ожидая ответа от устройства ввода-вывода, и не выполняет полезных вычислений.

Основная цель алгоритмов планирования – эффективное использование ресурсов центрального процессора. В данном случае под эффективностью понимается формирование очередности выполнения процессов таким образом, чтобы процессор максимальное время работал с задачами, ограниченными скоростью вычисления, и не простаивал при выполнении задач, ограниченных скоростью работы устройств ввода-вывода.

Эффективность планирования также зависит и от категории алгоритмов планирования:

1. Для пакетных систем эффективность алгоритмов планирования определяется количеством выполненных задач за единицу времени.
 2. Для интерактивных систем эффективность алгоритмов планирования определяется средним временем отклика на запрос пользователя.
 3. Для систем реального времени эффективность алгоритмов планирования определяется соблюдением предельных сроков выполнения задач и предотвращением потери данных.
- Рассмотрим подробнее различные алгоритмы планирования.

2.4.1 Алгоритмы планирования пакетных систем

Первым пришел – первым обслужен

Самым простым алгоритмом для пакетных систем является алгоритм «первым пришел – первым обслужен» (first in, first out, FIFO). При работе данного алгоритма задачи помещаются в очередь и выполняются в том же порядке. В некоторых случаях процесс, который обращается к устройству ввода-вывода, блокируется, удаляется из очереди и ожидает ответа от устройства ввода-вывода. После ответа от устройства ввода-вывода такой процесс переводится в состояние готовности и помещается в конец очереди. Некоторые реализации данного алгоритма не предполага-

ют удаление заблокированного процесса из очереди – процессор простаивает, ожидая ответа от устройства ввода-вывода.

Сначала самое короткое задание

Развитием алгоритма «первым пришел – первым обслужен» является алгоритм «сначала самое короткое задание».

Данный алгоритм основан на предположении, что известно время выполнения всех задач. Время выполнения может быть известно исходя из статистики выполнения подобных задач, накопленной в процессе работы системы. При добавлении новой задачи элементы очереди сортируются по времени выполнения в порядке возрастания. Недостатком данного алгоритма является возможность возникновения ситуации, при которой задача с максимальным временем исполнения никогда не будет исполнена.

Приоритет наименьшему времени выполнения

Алгоритм «приоритет наименьшему времени выполнения» похож на алгоритм «сначала самое короткое задание», но имеет следующее отличие – если при добавлении в очередь нового задания время его выполнения меньше оставшегося времени выполнения текущей задачи, то текущая задача прерывается и выполняется новая, более короткая задача.

2.4.2 Алгоритмы планирования интерактивных систем

Циклическое планирование

Алгоритм циклического планирования является базовым алгоритмом планирования для интерактивных систем. При циклическом планировании формируется очередь процессов, каждому процессу выделяется квант времени на выполнение, по истечению кванта времени текущий процесс прерывается и перемещается в конец списка, происходит смена контекста исполнения и выполняется следующий в очереди процесс. Если текущий процесс завершается или переходит в состояние блокировки, то незамедлительно происходит переход к следующему процессу, даже если квант времени текущего процесса не израсходован.

Размера кванта времени влияет на характер работы данного алгоритма:

- если квант времени будет меньше времени, необходимого для переключения контекста, то алгоритм будет работать неэффективно;
- если квант времени будет больше, чем время выполнения процессов, то работа алгоритма будет напоминать алгоритм «первым пришел – первым обслужен», что приведет к снижению времени отклика системы.

Обычно размер кванта времени задается в диапазоне от 20 до 50 мс.

Приоритетное планирование

Алгоритм приоритетного планирования основан на циклическом алгоритме, но при этом каждый процесс обладает некоторым приоритетом. Приоритет процесса влияет на квант времени прямо или косвенно:

- квант времени зависит от приоритета (прямо) – чем выше приоритет, тем больше квант времени;
- квант времени имеет фиксированный размер и не зависит от приоритета (косвенно). В данном случае алгоритм работает следующим образом:
 1. Процессы сортируются по значению приоритетов.
 2. Выполняется первый процесс.
 3. Выполняется снижение значение приоритета на одно значение.
 4. Если значение приоритета текущего процесса меньше значения приоритета следующего в очереди процесса, то происходит прерывание текущего процесса, восстановление его приоритета, перемещение текущего процесса в конец очереди.
 5. Осуществляется переход ко 2-у шагу алгоритма.

Использование нескольких очередей

При работе данного алгоритма формируется несколько очередей процессов. Новому процессу присваивается наивысший приоритет и процесс помещается в первую очередь, в которой каждому процессу выделяется 1 квант времени. Как только процесс использует все выделенные ему кванты времени его приоритет понижается, и он перемещается в следующую очередь с

увеличением количества выделенных квантов времени в два раза по сравнению с предыдущей очередью.

Выбор следующим самого короткого процесса

Данный алгоритм является адаптацией алгоритма планирования для пакетных систем «сначала самое короткое задание» к работе в интерактивных системах. Расчет времени выполнения процесса осуществляется через вычисление взвешенной суммы предыдущих значений времени выполнения процесса.

Гарантированное планирование

При использовании гарантированного планирования квант времени для каждого из n процессов будет равен $1/n$ от общего времени выполнения процессов за один цикл.

Лотерейное планирование

Лотерейное планирование предполагает наличие лотерейных билетов, которые выделяются процессам. При планировании случайным образом выбирается один из выданных процессам билетов. В результате выполняется тот процесс, чей лотерейный билет был выбран. Количество выделенных процессу лотерейных билетов повышает вероятность выполнения именно этого процесса. Например, процессу выделено 5% от общего числа лотерейных билетов, следовательно данный процесс может выиграть в лотерее с вероятностью равной 5%.

Справедливое планирование

При справедливом планировании учитывается принадлежность процесса пользователю. Квант времени выделяется таким образом, чтобы суммарное значение квантов времени для всех процессов текущего пользователя было равно суммарному значению квантов времени остальных пользователей системы. Каждый пользователь получает равное количество процессорного времени, которое распределяется между всеми его процессами.

2.4.3 Алгоритмы планирования систем реального времени

Алгоритмы планирования систем реального времени ориентированы на выполнение задач в строго отведенные промежутки времени, при этом получение верного результата с опозданием также неприемлемо, как и неполучение результата.

Системы реального времени подразделяются на

- системы жесткого реального времени, в которых соблюдение крайних сроков обязательно;
- системы мягкого реального времени, в которых несоблюдение крайних сроков нежелательно, но в некоторых случаях может быть допустимо.

Режим реального времени достигается за счет разделения программы на несколько процессов, поведение каждого из которых предсказуемо и заранее известно. Алгоритмы планирования систем реального времени должны спланировать выполнение процессов таким образом, чтобы суммарное время их выполнения не превышало заданных сроков с гарантированным получением результата.

2.5 Планирование потоков

Для планирования потоков обычно используются те же алгоритмы, что и для планирования процессов. Однако подход к планированию потоков может различаться в зависимости от модели реализации многопоточности: в пространстве ядра или в пространстве пользователя.

При реализации многопоточности в пространстве ядра единицей планирования являются потоки, которые планируются напрямую операционной системой с помощью алгоритмов планирования, рассмотренных ранее. Процессы в планировании не участвуют.

При реализации многопоточности в пространстве пользователя на уровне ядра операционной системы выполняется планирование процессов, а планирование потоков выполняется в пространстве пользователя в рамках кванта времени, выделенного процессу. При этом также могут использоваться алгоритмы планирования процессов, представленные выше.

2.6 Контрольные вопросы

1. Что такое процесс?
2. Что такое поток?
3. В чем различия между процессом и потоком?
4. Для чего необходимо планировать выполнение процессов/потоков?
5. Какие классы алгоритмов планирования существуют?
6. Перечислите алгоритмы планирования для пакетных систем.
7. Перечислите алгоритмы планирования для интерактивных систем.
8. В чем различия между потоками пространства ядра и пространства пользователя?
9. Что такое смена контекста?

3 УПРАВЛЕНИЕ ПАМЯТЬЮ

Память любой современной вычислительной системы (ВС) представляет собой совокупность различных типов памяти, обладающих различными характеристиками, местом расположения и особенностями работы с данными. На рисунке 3.1 представлена иерархия памяти ВС.



Рис. 3.1. Иерархия памяти ВС

Как видно из рисунка 3.1, память выстроена по мере снижения скорости ее работы и уменьшения стоимости за мегабайт. Самой быстрой памятью в ВС являются регистры центрального процессора, так как они располагаются непосредственно в тракте данных. При этом регистры обладают самым минимальным объемом из всех типов памяти ВС. Размер регистра обычно соответствует разрядности процессора, например, у 64-разрядных процессоров размер регистра составляет 64 бита. Скорость чтения данных из регистра не превышает времени выполнения одного цикла тракта данных.

Кэш-память также располагается в центральном процессоре ВС, но в зависимости от уровня (L1, L2, L3 и т. д.) кэша увеличивается и расстояние от тракта данных до такой памяти. Скорость работы кэш-памяти ниже, чем у регистров и уменьшается при увеличении уровня кэша. Также при увеличении уровня кэша увеличивается и объем соответствующей кэш-памяти. Например, в 10-м поколении процессоров семейства Intel Core размер кэша

L1 составляет 80 КБ для каждого вычислительного ядра, кэша L2 — 512 КБ для каждого вычислительного ядра, а общего кэша L3 — до 8 Мб. Скорость чтения данных кэша L1 может достигать 2344.8 ГБ/с, кэша L2 — 893.5 ГБ/с, а кэша L3 — 367.7 ГБ/с.

Кэш-память обычно используется для хранения данных, полученных из оперативной памяти ВС, к которым происходит частое обращение. Данные помещаются из оперативной памяти в кэш-память для сокращения времени доступа к ней. Также в кэш-память могут попадать данные, располагающиеся в некотором интервале адресов слева и справа от запрошенных данных (принцип локальности).

Оперативная память подключается в соответствующий разъем на системной плате. Таким образом для доставки данных из оперативной памяти в тракт данных требуются значительные временные затраты, например, скорость чтения данных памяти DDR4 может достигать 45.6 ГБ/с. Размер оперативной памяти в современных ВС может достигать нескольких сотен ГБ.

Оперативная память используется для хранения данных запущенных процессов, находящихся в состоянии выполнения, блокировки или ожидания. Фактически, оперативная память выступает некоторым аналогом кэш-памяти для данных, расположенных на различных устройствах хранения данных: твердотельные накопители, магнитные диски и т. д.

Устройства хранения данных обладают наибольшим объемом в иерархии памяти ВС, например, объем магнитных дисков может достигать нескольких десятков терабайт. Скорость доступа к данным, расположенным на устройствах хранения данных, является самой низкой, например, скорость чтения данных твердотельного накопителя может достигать 3.8 ГБ/с, а магнитного накопителя — 0.6 ГБ/с. В отличие от других типов памяти, устройства хранения данных являются энергонезависимой памятью и позволяют сохранять данные даже при отсутствии питания.

Низкая скорость работы устройств хранения данных обусловлена особенностями методов подключения к ВС и методов доступа к данным — требуется взаимодействие с контроллером устройства. В некоторых случаях на скорость работы с данными влияют конструктивные особенности устройства.

Таким образом, задача операционной системы — превратить рассмотренную иерархию памяти в набор абстракций и управлять данными абстракциями.

В рамках данной главы будут рассмотрены различные абстракции для работы с оперативной памятью ВС. Часть операционной системы, управляющая оперативной памятью, называется менеджером или диспетчером памяти.

3.1 Абстракции для работы с оперативной памятью

Основной целью абстракций операционной системы для работы с оперативной памятью, является предоставление программисту средств, позволяющих прозрачно управлять общим пространством оперативной памяти. Оперативная память в ВС, является общим разделяемым (в пространстве) ресурсом.

Основными проблемами, на решение которых в разной степени направлены абстракции для работы с оперативной памятью, являются;

1. Повышение безопасности работы пользовательских процессов в многозадачных операционных системах.
2. Защита области памяти операционной системы от воздействия со стороны пользовательских процессов в однозадачных системах.
3. Выделение свободной памяти.
4. Возможность использования больше памяти, чем позволяет объем физической памяти, установленной в ВС.
5. Решение проблемы внутренней и внешней фрагментации памяти (эффективное использование памяти).

3.1.1 Прямой доступ к памяти

Универсальные машины до начала 1960-х годов, миникомпьютеры до начала 1970-х годов и персональные компьютеры до начала 1980-х годов *не* использовали абстракции памяти. Данное обстоятельство связано с тем, что в таких ВС отсутствовала поддержка многозадачности:

- в универсальных машинах использовалась пакетная обработка данных, предполагающая поочередное исполнение заданий;

- в миникомпьютерах и персональных компьютерах было возможно запустить лишь один активный процесс.

Таким образом, отсутствовала необходимость разграничения памяти между процессами, так как один процесс целиком использовал всю доступную память.

При запуске процесса в режиме прямого доступа к памяти ему выделялась некоторая область памяти с диапазоном адресов, например, от 0 до некоторого максимума. Каждый адрес соответствовал ячейке памяти, содержащей некоторый набор бит.

При запуске процесса операционная система копирует запрошенные данные с диска в память, после чего процесс выполняется. При старте следующего процесса операционная система загружает в память данные нового процесса, записывая их поверх данных предыдущего процесса.

Предоставление процессам прямого доступа к памяти имеет ряд серьезных недостатков:

1. Если пользовательские программы могут обращаться к памяти напрямую, они легко могут преднамеренно или случайно испортить область памяти, принадлежащую операционной системе. Эта проблема присутствует даже при возможности запуска только одного активного процесса.
2. При использовании данной модели довольно сложно организовать одновременную (поочередную, если имеется лишь одно вычислительное ядро) работу нескольких процессов.

При прямом доступе к памяти для защиты области памяти, принадлежащей операционной системе, используется следующий подход:

1. Область памяти операционной системы непрерывно размещается либо в нижних адресах памяти (рис. 3.2.а), либо в верхних адресах памяти (рис. 3.2.б).
2. При обращении к памяти из пользовательского процесса происходит проверка на возможность доступа к определенному адресу.
3. Если запрашиваемый адрес не входит в область памяти операционной системы, доступ разрешается.
4. Если запрашиваемый адрес входит в область памяти операционной системы, генерируется системное прерывание.



Рис. 3.2. Организация памяти при наличии операционной системы и одного пользовательского процесса

Для ускорения изложенного выше метода защиты области памяти операционной системы обычно используется аппаратная реализация данной проверки на уровне центрального процессора.

3.1.2 Выделение свободной памяти

Для выделения свободной памяти для вновь созданного пользовательского процесса операционная система должна иметь сведения о занятых и свободных блоках оперативной памяти. Также эти сведения могут быть использованы для освобождения занятой памяти при завершении процесса.

Существует два метода хранения сведений о свободных и занятых блоках физической памяти:

1. Битовая матрица (рис. 3.3.б).
2. Связный список (рис. 3.3.в).

Как видно из рисунка 3.3.а, в физической памяти, имеющей 16 ячеек, запущено 4 пользовательских процесса, занимающих области памяти разной длины.

При использовании битовой матрицы (рис. 3.3.б) для хранения сведений о свободных и занятых блоках физической памяти

необходимо определить размер блока физической памяти. От размера блока будет зависеть количество ячеек битовой матрицы. Занятые блоки физической памяти могут быть помечены в битовой матрице как 1, а свободные как 0 (или наоборот).

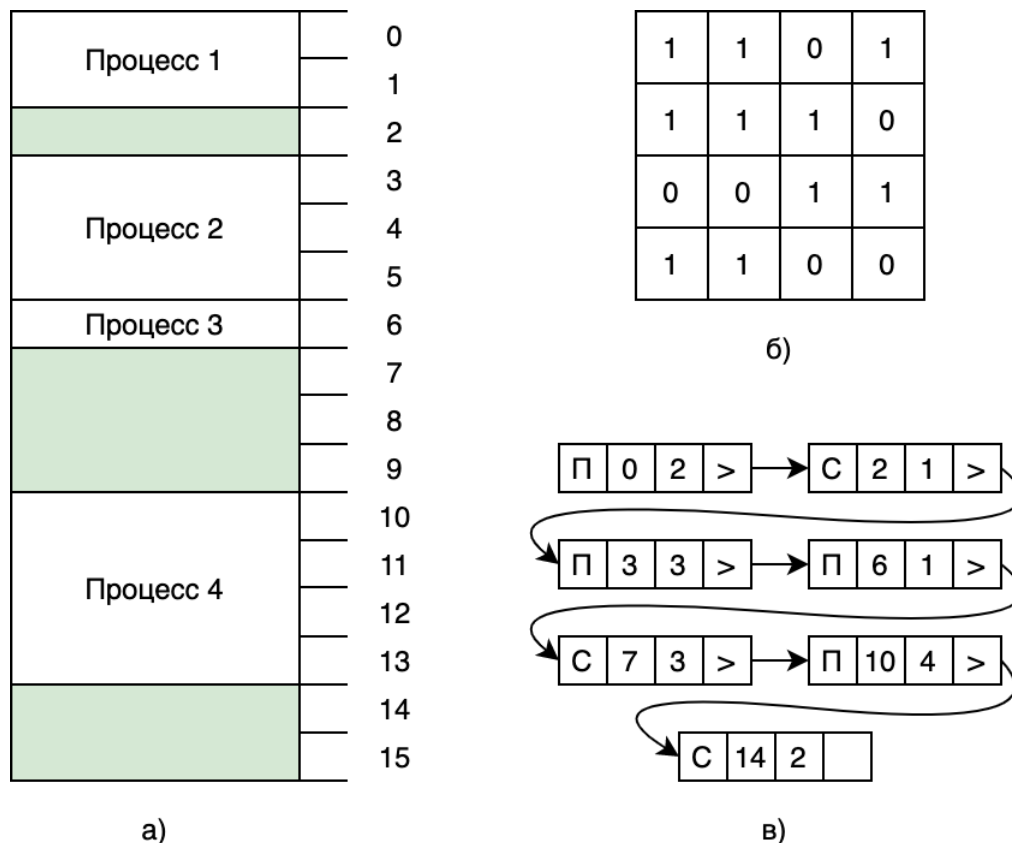


Рис. 3.3. Иллюстративный пример хранения сведений о свободных и занятых блоках физической памяти

Преимуществом использования битовой матрицы для хранения сведений о свободных и занятых блоках физической памяти является простота реализации, недостатком — низкая скорость поиска свободного пространства необходимой длины.

При использовании связанного списка (рис. 3.3.в) для хранения сведений о свободных и занятых блоках физической памяти каждый элемент такого списка обычно содержит следующие сведения:

- тип элемента: область памяти, занятая процессом (П), или свободная область памяти (С);
- начальный адрес области в физической памяти;
- размер области в физической памяти;
- ссылка на следующий элемент связанного списка.

Связный список обычно отсортирован по размеру области. Таким образом, скорость поиска свободного пространства необходимой длины будет выполняться быстрее, чем при использовании битовой матрицы. Недостатком связного списка является более сложная реализация такого подхода.

3.1.3 Алгоритмы поиска свободного блока памяти

Для поиска свободного блока памяти необходимой длины обычно используются вариации следующих алгоритмов:

- первое подходящее. Диспетчер памяти осуществляет поиск до тех пор, пока не найдет пустое пространство подходящего размера. Затем пустое пространство разбивается на две части: одна для процесса и одна остается пустой;
- следующее подходящее. Данный алгоритм аналогичен алгоритму «первое подходящее». Отличием является учет местоположения — как только диспетчер памяти находит подходящее пустое пространство данная позиция запоминается. При следующей работе алгоритма поиск осуществляется с того места, на котором алгоритм остановился в прошлый раз, а не приступает к поиску с самого начала;
- наиболее подходящее. Данный алгоритм выбирает пустое пространство наиболее близкое по размеру к искомому месту разбиения пустого пространства на более мелкие области;
- наименее подходящее. Данный алгоритм выбирает пустое пространство наибольшей длины, чтобы вновь образующееся пустое пространство было достаточно большим для дальнейшего использования;
- быстро искомое подходящее. При работе данного алгоритма предусматривается ведение отдельных списков для некоторых наиболее востребованных искомых размеров, например, 2 КБ, 4 КБ и т. д.

3.1.4 Проблема нехватки физической памяти

Проблема запуска программ, превышающих по объему размер имеющейся физической памяти, возникла достаточно давно. В 60-е годы XX века было принято решение разбивать программы на небольшие части, называемые оверлеями. При запуске про-

граммы в память загружался только администратор оверлейной загрузки, который тут же загружал и запускал оверлей с порядковым номером 0. Когда этот оверлей завершал свою работу, он мог сообщить администратору загрузки оверлеев о необходимости загрузки оверлея 1.

Некоторые оверлейные системы имели довольно сложное устройство, позволяя одновременно находиться в памяти множеству оверлеев. Оверлеи хранились на диске, и их свопинг с диска в память и обратно осуществлялся администратором загрузки оверлеев.

Работа по свопингу оверлеев с диска в память и обратно выполнялась операционной системой, при этом разбиение программ на части выполнялось программистом в ручном режиме. Разбиение больших программ на небольшие модульные части было очень трудоемкой, скучной и не застрахованной от ошибок работой.

3.1.5 Внутренняя и внешняя фрагментация

Под внутренней фрагментацией понимается выделение пользовательскому процессу большего объема физической памяти, чем ему требуется. В результате оперативная память используется неэффективно.

Под внешней фрагментацией понимается ситуация, когда свободная память имеется в нужном объеме, но разделена на несколько блоков меньше длины, чем необходимо пользовательскому процессу. Для решения проблемы внешней фрагментации обычно используется дефрагментация — уплотнение памяти пользовательских процессов.

Процесс дефрагментации можно представить в виде последовательности следующих шагов:

1. Выгрузить область памяти простаивающего в данный момент пользовательского процесса на диск.
2. Загрузить область памяти по другому адресу.
3. Скорректировать значение сведений о расположении области памяти пользовательского процесса согласно новому положению в физической памяти.

3.1.6 Адресное пространство

При добавлении в операционную систему многозадачности — возможности одновременного запуска нескольких пользовательских процессов, появилась необходимость в изоляции областей памяти разных пользовательских процессов от преднамеренного или случайного воздействия со стороны других процессов. Для решения проблемы изоляции областей памяти пользовательских процессов используется адресное пространство.

Адресное пространство — набор адресов, который может быть использован пользовательским процессом для обращения к памяти. У каждого процесса имеется свое собственное адресное пространство, независимое от того адресного пространства, которое принадлежит другим процессам (за исключением тех особых обстоятельств, при которых процессам требуется совместное использование их адресных пространств).

На рисунке 3.4 представлен пример адресного пространства двух пользовательских процессов: «Процесс 1» и «Процесс 2» соответственно.

0	16380	0
	...	
MOV	16	
MOV	12	ADD
	8	MOV
	4	
JMP 12	0	JMP 8
Процесс 1		Процесс 2

Рис. 3.4. Иллюстративный пример адресного пространства

Как видно из рисунка 3.4, пользовательские процессы «Процесс 1» и «Процесс 2» имеют одинаковый размер адресного пространства — 16380. Использование изолированных адресных пространств формирует так называемую виртуальную память. При использовании виртуальной памяти пользовательские процессы для доступа к памяти используют виртуальные адреса, диапазон которых ограничен размером их адресного пространства. Адрес каждого адресного пространства начинается с 0, а адресация происходит в рамках доступных адресов от 0 до 16380. Виртуальные

адреса различных процессов могут пересекаться, так как они учитывают контекст отдельного адресного пространства — фактически, адресное пространство позволяет скрыть от пользовательского процесса наличие других пользовательских процессов.

Для работы с физической памятью и физическими (реальными) адресами используются различные подходы трансляции виртуальных адресов в физические адреса. Некоторые подходы к трансляции виртуального адреса в физический будут рассмотрены более подробно в следующих разделах.

После изоляции областей памяти пользовательских процессов с помощью непересекающихся адресных пространств необходимо решить проблему размещения нескольких адресных пространств в оперативной памяти ВС.

3.1.7 Свопинг

Самым простым способом организации работы с памятью с применением виртуальной памяти является свопинг.

При использовании свопинга адресное пространство пользовательского процесса полностью располагается в оперативной памяти. При переходе процесса в состояние выполнения адресное пространство предыдущего активного процесса сохраняется на устройстве хранения (диске), а адресное пространство текущего процесса загружается в оперативную память с диска. Адресные пространства бездействующих процессов, находящихся в состоянии блокировки или ожидания, времени хранятся на диске и не занимают пространство оперативной памяти.

К преимуществам свопинга можно отнести простоту реализации, а к недостаткам — медленную скорость работы из-за частого обращения к устройствам хранения при планировании процессов (переключении задач).

3.1.8 Базовый и ограничительный регистры

Для одновременного размещения в оперативной памяти адресных пространств нескольких пользовательских процессов используется подход с применением базового и ограничительного регистров.

Базовый и ограничительный регистры — регистры центрального процессора специального назначения. Базовый и ограничи-

тельный регистры являются классическим решением проблемы трансляции адресов — адресные пространства пользовательских процессов загружаются в последовательно расположенные свободные области памяти без модификации адресов в процессе загрузки.

При запуске процесса в базовый регистр записывается физический адрес, с которого начинается размещение адресного пространства процесса в физической памяти, а в ограничительный регистр записывается длина адресного пространства. Длина адресного пространства может быть фиксированной и неизменной, тогда процессы будут иметь адресные пространства типовой длины. Также длина адресного пространства может формироваться динамически на основе запроса пользовательского процесса, при этом выделенный размер адресного пространства по прежнему остается неизменным.

Использование базового и ограничительного регистров — простой способ предоставления каждому процессу своего собственного закрытого адресного пространства. Многие реализации предусматривают такую защиту базового и ограничительного регистров, при которой изменить их значения может только операционная система.

При каждом обращении процесса к памяти центральный процессор осуществляет трансляцию адресов:

1. Перед выставлением адреса на шине памяти к виртуальному адресу прибавляется значение базового регистра. Происходит формирование физического адреса.
2. Полученный физический адрес проверяется на выход за границы адресного пространства путем сравнения со значением ограничительного регистра. Если полученный физический адрес больше или равен значению ограничительного регистра, генерируется системное прерывание.

Рассмотрим процесс трансляции адресов с применением базового и ограничительного регистров более подробно. На рисунке 3.5 представлен фрагмент физической памяти, содержащей адресные пространства двух процессов: «Процесс 1» и «Процесс 2».

Как видно из рисунка 3.5, адресное пространство процесса «Процесс 1» начинается с адреса 0, а его длина составляет 16380. Следовательно, значение базового регистра для процес-

са «Процесс 1» будет 0, а значение ограничительного регистра — 16380, для процесса «Процесс 2» значение базового регистра будет 16384, а значение ограничительного регистра — 32764.

Процесс 2	0	32764
		...
		16400
	ADD	16396
	MOV	16392
		16388
	JMP 8	16384
	0	16380
		...
		20
Процесс 1	MOV	16
	MOV	12
		8
		4
	JMP 12	0

Рис. 3.5. Иллюстративный пример физической памяти, содержащей адресные пространства двух процессов

Допустим, что в процессе «Процесс 2» происходит выполнение команды с виртуальным адресом 0 — JMP 8. К виртуальному адресу 0 прибавляется значение базового регистра 16384, тогда физический адрес будет иметь значение 16384. Полученный физический адрес 16384 не выходит за границы адресного пространства. Команда JMP 8 выполняется, происходит безусловный переход к виртуальному адресу 8.

К виртуальному адресу 8 прибавляется значение базового регистра 16384, полученный физический адрес имеет значение 16392, данный адрес является допустимым. Происходит выполнение команды, расположенной по физическому адресу 16392 (виртуальный адрес 8) MOV и т. д.

Преимущества использования базовых и ограничительных регистров:

- простота реализации;
- отсутствие внутренней фрагментации при динамическом определении размера адресного пространства.

Недостатками использования базовых и ограничительных регистров являются:

- требуется аппаратная поддержка;
- необходимость применения операций сложения и сравнения к каждой ссылке на ячейку памяти (сложение является слишком дорогой операцией);
- наличие внутренней и внешней фрагментации при фиксированном размере адресного пространства;
- сильно выраженная внешняя фрагментация при динамическом определении размера адресного пространства.

При использовании базового и ограничительного регистров решается проблема изоляции адресных пространств для реализации возможности одновременного размещения в оперативной памяти нескольких пользовательских процессов одновременно. Но такие проблемы как: внутренняя и внешняя фрагментация, выделение свободной памяти и нехватка физической памяти остаются нерешенными.

3.1.9 Страничная организация виртуальной памяти

В основе виртуальной памяти со страничной организацией лежит идея, что у каждого процесса имеется свое собственное адресное пространство, которое разбивается на участки, называемые виртуальными страницами. Виртуальные страницы отображаются на страницы физической памяти. Каждая страница представляет собой непрерывный диапазон адресов. Для работоспособности процесса присутствие в памяти всех его виртуальных страниц не обязательно.

Страница обычно имеет размер 4 КБ, но в зависимости от операционной системы размер страницы может отличаться. Размер страницы обязательно должен быть кратен 2.

Для хранения информации о виртуальных страницах используется таблица страниц. Ссылка на таблицу страниц содержится в таблице процессов. Запись таблицы страниц обычно содержит следующие сведения:

- идентификатор виртуальной страницы;
- признак присутствия виртуальной страницы в физической памяти;

- признак модификации страницы — используется в процессе подкачки страниц для определения необходимости обновления копии страницы на диске (раздел или файл подкачки);
- признак обращения — используется в работе алгоритмов замещения страниц (будет рассмотрено в следующем разделе);
- права доступа — определяет возможность чтения и/или записи данной страницы;
- признак блокирования кэширование — отключение кэширования страницы;
- номер страницы физической памяти, на которую отображена данная виртуальная страница.

На рисунке 3.6 представлен иллюстративный пример отображения страниц виртуальной памяти двух процессов на страницы физической памяти. На рисунке 3.6 также представлено содержимое таблицы страниц для каждого из двух процессов.

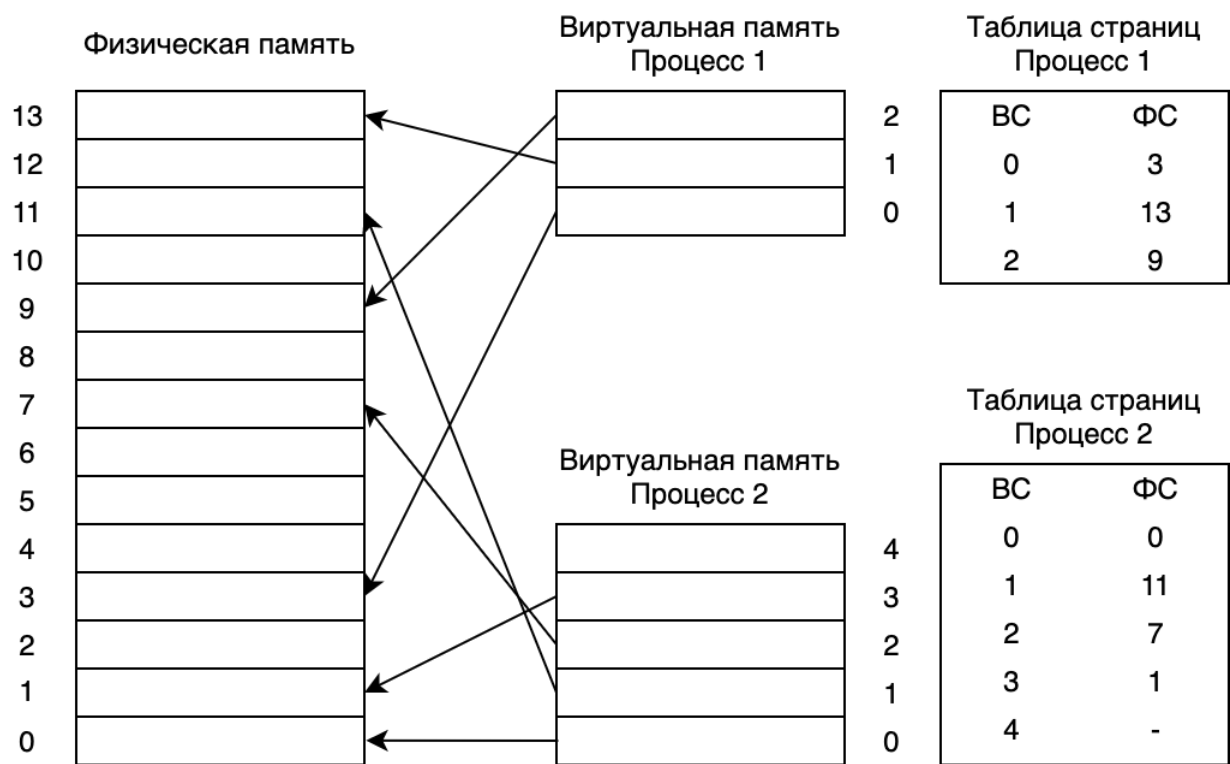


Рис. 3.6. Иллюстративный пример отображения страниц виртуальной памяти двух процессов на страницы физической памяти

Для повышения скорости работы виртуальной памяти со страничной организацией используется аппаратный буфер ассоциативной трансляции адресов (Translation lookaside buffer, TLB). TLB — это специализированный кэш центрального процессора,

используемый для ускорения трансляции адреса виртуальной памяти в адрес физической памяти. Существует возможность программного обновления индекса TLB в случае отсутствия страницы в TLB и ее наличия в физической памяти.

Когда в процессе происходит обращение к некоторому участку адресного пространства, отображенного на физическую память, центральный процессор осуществляет трансляцию адресов. Если некоторая страница, к которой было совершено обращение, *не* находится в физической памяти:

1. Операционная система генерирует страничное прерывание (page fault).
2. Текущий процесс прерывает свою работу.
3. Операционная система производит отображение виртуальной страницы на свободную физическую страницу.
4. Операционная система модифицирует таблицу страниц текущего процесса.
5. Текущий процесс продолжает свою работу.

Иллюстративный пример процесса трансляции виртуального адреса в физический представлен на рисунке 3.7.

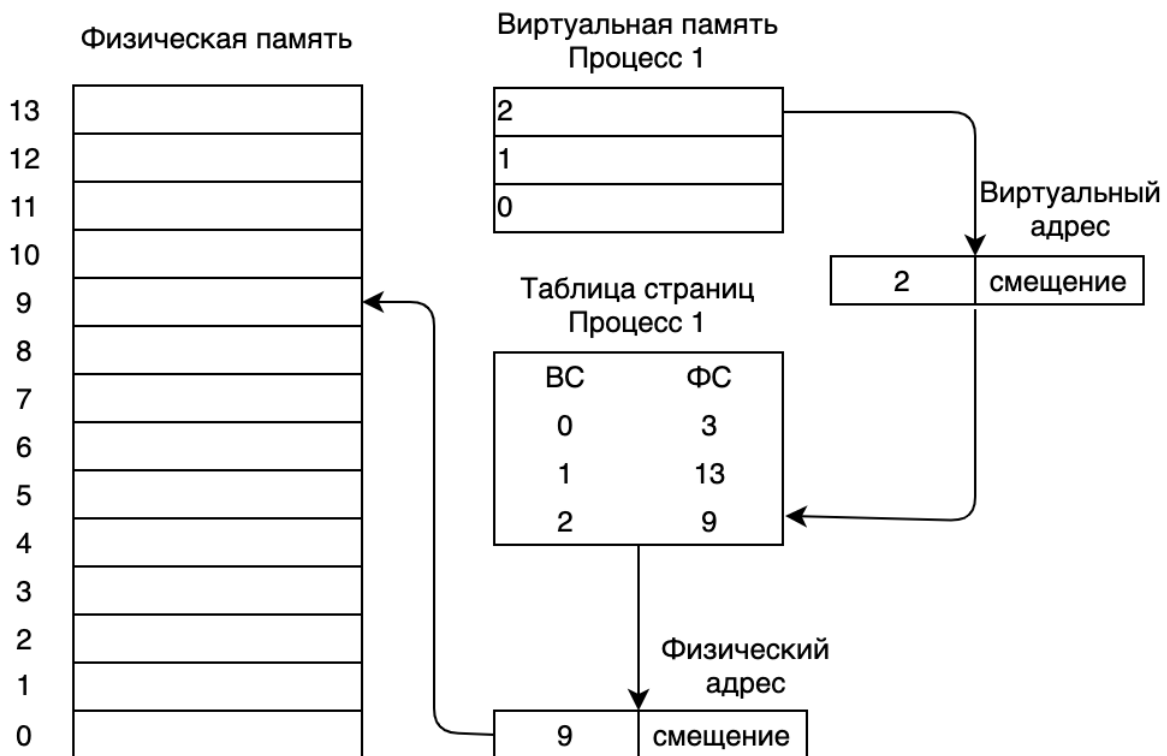


Рис. 3.7. Иллюстративный пример трансляции виртуального адреса в физический

Как видно из рисунка 3.7, виртуальный адрес состоит из двух значений: идентификатор виртуальной страницы и смещение относительно начала страницы. Смещение используется для позиционирования внутри страницы.

Затем в таблице страниц происходит поиск необходимой записи по идентификатору виртуальной таблицы. Если запись существует то она извлекается и обрабатывается, иначе создается запись для новой виртуальной страницы.

После получения записи таблицы страниц происходит проверка присутствия страницы в физической памяти, если виртуальная страница отображена на физическую память, происходит извлечение физического адреса из записи таблицы страниц. Иначе происходит страничное прерывание и отображение новой виртуальной страницы на физическую память с последующей установкой признака присутствия страницы и получения ее физического адреса.

Далее полученный физический адрес используется для обращения к странице физической памяти.

Также виртуальная память со страничной организацией позволяет выделить процессам больше памяти, чем доступно физической памяти. Данная возможность реализуется за счет выгрузки неиспользуемых в данный момент страниц в файл или раздел подкачки, который находится на диске. Подкачка страниц очень похожа на процесс свопинга — некоторые страницы при необходимости выгружаются на диск, а другие страницы загружаются в оперативную память.

В процессе подкачки используется признак модификации страницы таблицы страниц. Если некоторую страницу необходимо выгрузить на диск и ее модификация не производилась, тогда страница просто удаляется из физической памяти, иначе происходит запись выгружаемой страницы на диск. При удалении отображения виртуальной страницы на страницу физической памяти признак присутствия страницы в физической памяти сбрасывается.

При обращении к выгруженной на диск странице, она вновь загружается в свободный блок физической памяти и происходит обновление ее записи таблицы страниц: устанавливается признак присутствия, обновляется номер страницы физической памяти.

Таким образом, виртуальная таблица со страничной организацией решает практически все поставленные проблемы управления памятью:

1. Безопасность работы пользовательских процессов в многозадачных системах обеспечена наличием изолированных адресных пространств.
2. Выделение свободной памяти заключается в поиске любого свободного пространства, так как все страницы физической памяти обычно имеют одинаковый размер.
3. При использовании страничной организации появляется возможность помещать в физическую память только те страницы, которые необходимы процессу в настоящий момент времени. Появляется возможность использовать больше памяти, чем позволяет объем установленной физической памяти.
4. Внешняя фрагментация полностью устраняется за счет одинакового размера страниц.
5. Влияние внутренней фрагментации на эффективность использования памяти значительно уменьшается за счет достаточно небольшого размера страниц.

К недостаткам виртуальной памяти со страничной организацией можно отнести следующие пункты:

1. Высокие накладные расходы при обращении к памяти.
2. Большой объем памяти требуется для хранения множества таблиц страниц.
3. Высокие накладные расходы на подкачку и замещение страниц.

3.1.10 Сегментная и сегментно-страничная организация виртуальной памяти

Для решения многих проблем наличие двух и более отдельных виртуальных адресных пространств может быть более рациональным вариантом, чем наличие только одного адресного пространства.

Сегментная организация памяти позволяет разбивать адресное пространство на сегменты — логические блоки, например, блок кода, блок данных, куча, стек и т. д. При использовании сег-

ментной организации памяти виртуальный адрес содержит номер сегмента и смещение относительно начала сегмента.

Каждый сегмент представляет собой последовательность адресов от 0 до определённого максимального значения. Отличие сегмента от страницы заключается в возможности изменения размера сегмента в процессе работы. Длина каждого сегмента может иметь любое значение от 0 до максимально разрешенного. Длина сегмента может увеличиваться при поступлении в него данных и уменьшаться при их извлечении из него. Сегменты могут размещаться как в оперативной памяти, так и на диске.

Сегментно-страничный метод организации памяти объединяет достоинства сегментного и страничного метода организации памяти. При сегментно-страничной организации памяти адресное пространство разбивается на ряд сегментов. Каждый сегмент состоит из страниц физической памяти. Виртуальный адрес также состоит из номера сегмента и смещения относительно начала сегмента.

Сегментная и сегментно-страничная организация памяти требует наличия более сложной аппаратной и программной составляющей.

3.2 Алгоритмы замещения страниц

Необходимость в замещении страниц возникает в том случае, если при отображении виртуальной страницы на страницу физической памяти в оперативной памяти закончилось свободное пространство. В таком случае необходимо определить какие страницы физической памяти являются наименее востребованными и выгрузить их на диск (осуществить подкачку).

Для решения данной задачи используются различные алгоритмы замещения страниц. Сложность реализации алгоритмов замещения страниц заключается в том, что невозможно предсказать, к какой странице будет обращение в некоторый момент времени.

Обобщенно схему работы алгоритмов замещения страниц можно представить в виде последовательности шагов:

1. Определить страницы, которые не будут востребованы в ближайшее время.

2. Из полученного множества страниц попытаться найти страницу, которая не была модифицирована и удалить ее из физической памяти. Данная проверка позволяет обойтись без записи страницы на диск.
3. Загрузить другую страницу в освободившийся блок физической памяти.

Основной целью любого алгоритма замещения страниц является уменьшение числа страничных прерываний путем определения лучшего кандидата для выгрузки — страницы, к которой гарантированно не будет произведено обращений в ближайший максимальный период времени. Идеальный вариант — страница, к которой вообще не будет обращений.

Рассмотрим некоторые алгоритмы замещения страниц более подробно.

3.2.1 *Оптимальный алгоритм*

Оптимальный алгоритм замещения страниц позволяет найти самую оптимальную для выгрузки страницу. Данный алгоритм невозможно реализовать для работы в реальных условиях, так как он требует наличия сведений о будущем состоянии всех страниц.

Оптимальный алгоритм можно реализовать в некоторой модельной системе, в которой все события предопределены и известны заранее. Обычно результаты работы подобной реализации оптимального алгоритма используются для сравнения с результатами работы других алгоритмов замещения страниц, которые не обладают сведениями о развитии событий и работают в режиме приближенном к реальному.

Сложно определить наилучший и наихудший алгоритм замещения страниц, так как в зависимости от множества условий разные алгоритмы могут показывать совершенно разные результаты.

3.2.2 *Алгоритм FIFO*

Самым простым в реализации алгоритмом замещения страниц является алгоритм FIFO (First-In, First-Out). Данный алгоритм работает с очередью страниц. Новая страница помещается в конец очереди, а выгружается первая в очереди страница.

К преимуществам данного алгоритма можно отнести выгрузку самой старой страницы, а к недостаткам — отсутствие уверенности в том, что самая старая страница не используется.

В большинстве случаев при использовании данного алгоритма общая производительность ВС является достаточно низкой.

3.2.3 Алгоритм «Второй шанс»

Алгоритм «Второй шанс» основан на алгоритме FIFO, но использует признак обращения для определения выгружаемой страницы:

1. Если у страницы, находящейся в начале очереди, установлен признак обращения, то значение признака сбрасывается, а страница перемещается в конец очереди.
2. Если у страницы, находящейся в начале очереди, признак обращения не установлен, то страница выгружается.

В отличие от алгоритма FIFO, алгоритм «Второй шанс» никогда не выгрузит востребованную страницу.

3.2.4 Алгоритм LRU

Алгоритм LRU (Least Recently Used) пытается найти наименее востребованную страницу, используя для этого признак обращения таблицы страниц. При реализации данного алгоритма в признак обращения записывается временная метка доступа к странице. Затем страницы сортируются в порядке возрастания данного значения, выгружается страница, к которой давно не было обращений.

К недостаткам данного алгоритма можно отнести необходимость хранения и обработки временных меток, что является достаточно трудоемкой задачей.

3.2.5 Алгоритм NRU

Алгоритм NRU (Not Recently Used) пытается найти страницу, к которой не было обращений в последнее время. При работе данного алгоритма используется признак модификации и признак обращений таблицы страниц.

Признак обращений устанавливается в 1 при обращении и сбрасывается в 0 через некоторый промежуток времени. Признак модификации устанавливается в 1 только при модификации стра-

ницы. Таким образом, можно получить следующие классы страниц:

1. Не было обращений и модификаций.
2. Не было обращений, была модификация.
3. Было обращение, не было модификаций.
4. Было обращение и была модификация.

После классификации страниц, алгоритм выгружает произвольную страницу, относящуюся к самому низкому непустому классу — лучше выгрузить модифицированную страницу, чем страницу, к которой недавно было обращение.

Алгоритм NRU прост в реализации и позволяет добиться приемлемой производительности.

3.2.6 Алгоритм часы

При использовании для замещения страниц алгоритма часы страницы организуются в некое подобие циферблата. Часовая стрелка выбирает страницу для выгрузки, учитывая признак обращения:

- если признак обращения сброшен, страница выгружается;
- если признак обращения установлен, то его значение сбрасывается и происходит переход к следующей странице.

Алгоритм часы выгружает достаточно старую страницу. Новая страница загружается на место выгруженной страницы, и будет обработана алгоритмом только при прохождении стрелкой полного цикла.

3.2.7 Рабочий набор

При работе алгоритма рабочий набор формируется множество страниц, к которыми было обращение за некоторый прошедший интервал времени. Таким образом, состав страниц рабочего набора может изменяться во времени, как и длина рабочего набора.

Выгружаются страницы, не входящие в рабочий набор.

3.2.8 Алгоритм WSClock

Алгоритм WSClock основан на алгоритме часы, но также учитывает вхождение страницы в рабочий набор и признак модификации.

В процессе работы алгоритма модифицированные и востребованные страницы не являются кандидатами на выгрузку. Модифицированные страницы планируются для записи на диск, если стрелка часов указывает на них. Для выгрузки алгоритм пытается найти страницу, к которой не было обращений, которая не входит в рабочий набор и не была модифицирована.

Алгоритм WSClock используется на практике и хорошо себя зарекомендовал.

3.2.9 Алгоритм Page Fault Frequency

Алгоритм Page Fault Frequency в своей работе учитывает частоту страничных прерываний. Данный алгоритм пытается нормализовать и снизить количество страничных прерываний среди всего множества запущенных пользовательских процессов.

Принцип работы алгоритма Page Fault Frequency:

1. Определение числа страничных прерываний для каждого процесса.
2. Если количество страничных прерываний какого-либо процесса больше некоторого порогового значения, то такому процессу выделяется больше физической памяти.
3. Если количество страничных прерываний какого-либо процесса меньше некоторого порогового значения, то часть страниц такого процесса выгружается из физической памяти.

3.2.10 Пробуксовка

Состояние пробуксовки возникает, когда диспетчер памяти большую часть времени обслуживает страничные прерывания, что приводит к значительному снижению производительности ВС.

Состояние пробуксовки может возникнуть в случае:

- некорректной работы алгоритма замещения страниц;
- некорректной работы алгоритма замещения страниц при достаточном объеме физической памяти;
- наличия большого числа активных процессов и недостатке физической памяти.

3.3 Контрольные вопросы

1. Для чего используются базовый и ограничительный регистр?
2. Для чего используется свопинг?
3. Для чего используются оверлеи?
4. Какие проблемы решает виртуальная память?
5. Для чего используется страничная организация памяти?
6. Для чего используется сегментная организация памяти?
7. Что такое страничное прерывание?
8. Как работает трансляция адресов?
9. Для чего используются алгоритмы замещения страниц?
10. Перечислите и сравните алгоритмы замещения страниц.
11. Перечислите и сравните алгоритмы выделения памяти.

4 ФАЙЛОВЫЕ СИСТЕМЫ

В отличие от работы с энергозависимой оперативной памятью, при работе с энергонезависимой памятью — устройствами хранения данных — операционная система должна решать следующие задачи:

1. Предоставление возможности хранения любого объема информации.
2. Информация должна сохраняться даже после прекращения работы использующего ее процесса.
3. К информации должны иметь параллельный доступ несколько процессов.

Представленные выше задачи, в рамках современных операционных систем обычно решаются с помощью специальной абстракции — файла.

Файлы представляют собой логический информационный блок, позволяющий организовать связь между некоторой сущностью, с которой работает пользователь (и пользовательский процесс), и множеством физических блоков памяти, расположенных на устройствах хранения данных.

Также, операционная система позволяет создавать специализированные типы файлов — каталоги. Каталоги используются для организации иерархии файлов и каталогов.

Для управления файлами и каталогами, их атрибутами, правами доступа к информации, свободным пространством устройств хранения, способом организации логических и физических данных применяются различные файловые системы.

4.1 Файлы

Под файлом обычно понимается именованная область данных некоторого устройства хранения.

С функциональной точки зрения файлы подразделяются на:

- обычные файлы. Используются для хранения данных пользовательского процесса;
- системные файлы. Используются для хранения данных операционной системы и системных процессов;
- каталоги. Используются для организации иерархии файлов.

В некоторых операционных системах, например, в UNIX подобных операционных системах, файлы могут также представлять различные устройства ввода-вывода:

1. Символьные специальные файлы — используются для взаимодействия с последовательными устройств ввода-вывода, например, терминалы, принтеры и т. д. Символьные файлы позволяют получать и/или отправлять данные (символы) в последовательное устройство ввода-вывода.
2. Блочные специальные файлы — используются для взаимодействия с устройствами хранения данных. Блочные файлы позволяют как читать и/или записывать данные на устройство хранения, так и осуществлять позиционирование по содержимому устройства хранения.

Кроме того, обычные файлы также можно разделить на:

- текстовые файлы (ASCII) — двоичный файл, содержащий данные в текстовом формате. Текстовые файлы содержат последовательность символов, принадлежащих некоторому набору символов (кодировка). Текстовые файлы могут быть прочитаны любым текстовым редактором на любой операционной системе (при условии поддержки кодировки). При повреждении части текстового файла, остальные данные файла будут доступны.
- двоичные (бинарные) файлы — файл, имеющий определенную структуру для организации хранения данных. Для работы с двоичным файлом необходимо знать его структуру. При повреждении двоичного файла получить его содержимое может быть невозможно. Двоичные файлы позволяют более эффективно хранить данные, например, при использовании сжатия.

На рисунке 4.1 представлены примеры структуры двоичных файлов: исполняемый файл (рис. 4.1.а) и архив (рис. 4.1.б).

Для работы с файлами операционные системы обычно предоставляют следующие системные вызовы:

- Create — создание пустого файла с установкой некоторых его атрибутов;
- Delete — удаление файла;

- Open — открытие файла. При открытии файла создается запись в таблице открытых файлов, также происходит чтение атрибутов файла и их запись в оперативную память;
- Close — закрытие файла. При закрытии файла удаляется соответствующая запись из таблицы открытых файлов;
- Read — чтение данных из файла с текущей позиции;
- Write — запись данных в файл с текущей позиции;
- Append — запись данных в конец файла;
- Seek — перемещение указателя файла на определенную позицию;
- Get attributes — получение атрибутов файла;
- Set attributes — установка атрибутов файла;
- Rename — переименование файла.

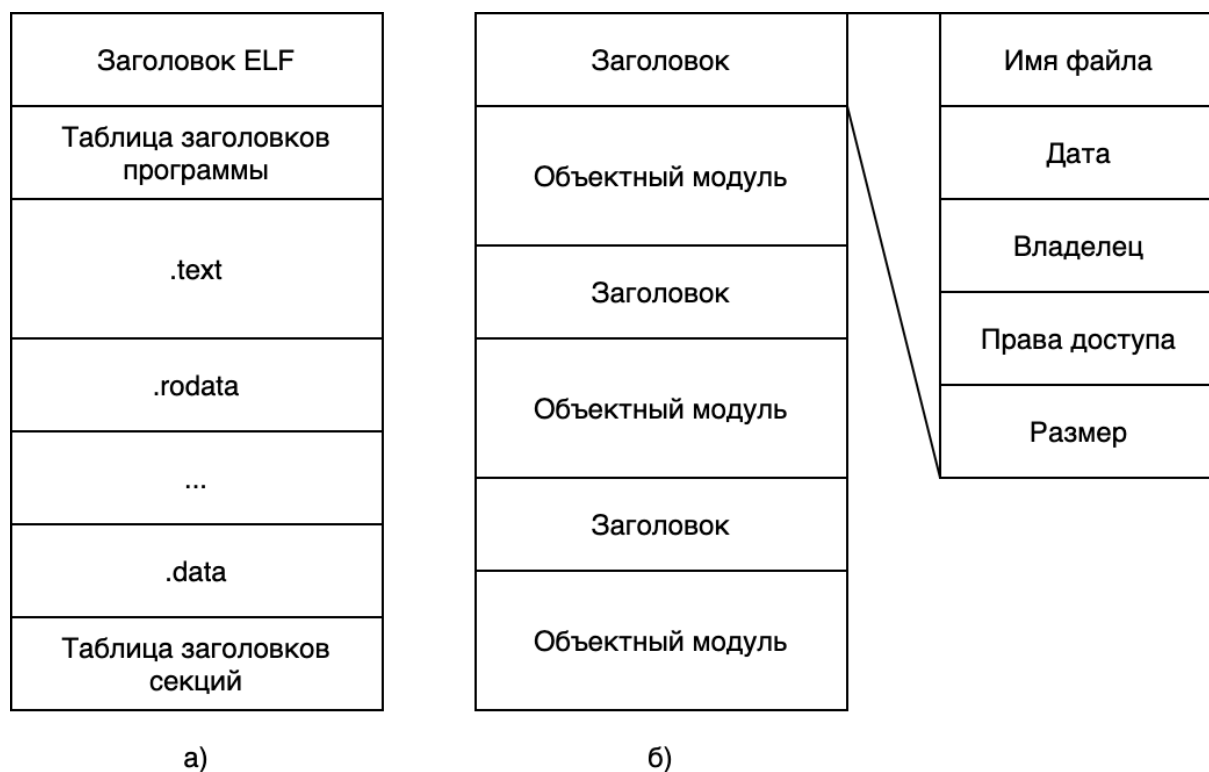


Рис. 4.1. Пример структуры двоичных файлов

4.2 Каталоги

Каталоги представляют собой специализированные типы файлов. Каталоги используются для группировки файлов и организации иерархии файлов и каталогов. Фактически, каталог со-

держит в себе перечень ссылок на файлы и каталоги, которые располагаются в нем.

Первые операционные системы имели только один каталог — корневой, без возможности создания новых каталогов. Соответственно, при увеличении количества приложений, пользователей и производных файлов, появилась необходимость разграничения файлов одного приложения и/или пользователя от других файлов.

Для работы с каталогами операционные системы обычно предоставляют следующие системные вызовы:

- Create — создание пустого каталога. Элементы . и .. создаются автоматически (см. далее);
- Delete — удаление пустого каталога. Для удаления каталога сначала необходимо удалить его содержимое;
- Opendir — открытие каталога;
- Closedir — закрытие каталога;
- Readdir — чтение содержимого каталога;
- Link — привязка существующего файла к каталогу с применением механизма жестких ссылок (hard link);
- Unlink — отвязка существующего файла от каталога. Если файл привязан только к одному каталогу, то он удаляется из файловой системы.

Символические и жесткие ссылки позволяют размещать один и тот же файл в нескольких директориях — файл не меняет своего изначального адреса, но к нему можно обратиться по ссылке, не зная его изначального адреса.

Символическая ссылка содержит адрес нужного файла или каталога в файловой системе. При обращении к файлу через символическую ссылку открывается целевой файл или каталог. Главное отличие символической ссылки от жестких ссылок заключается в том, что при удалении целевого файла и/или каталога ссылка остается в файловой системе, но переход по ней осуществляться не будет. Адреса символических ссылок могут быть относительными и абсолютными (см. далее).

Жесткие ссылки реализованы на более низком уровне файловой системы. При использовании жестких ссылок данные размещаются в единственном экземпляре в определенном месте устройства хранения данных. Но на эти данные могут ссылаться несколько жестких ссылок, имеющих различные адреса. При

этом, каждая жесткая ссылка представляет собой отдельный файл, ссылающийся на одни данные. При перемещении таких файлов между каталогами все ссылки остаются рабочими.

4.3 Адресация

Для получения доступа к файлу необходимо знать его адрес. Адресация пути файла бывает:

- абсолютной — путь к файлу указывается, начиная с корневого каталога и далее вглубь по дереву каталогов до требуемого файла;
- относительной — путь к каталогу указывается относительно адреса текущего каталога.

Рассмотрим примеры абсолютной и относительной адресации на примере рисунка 4.2.

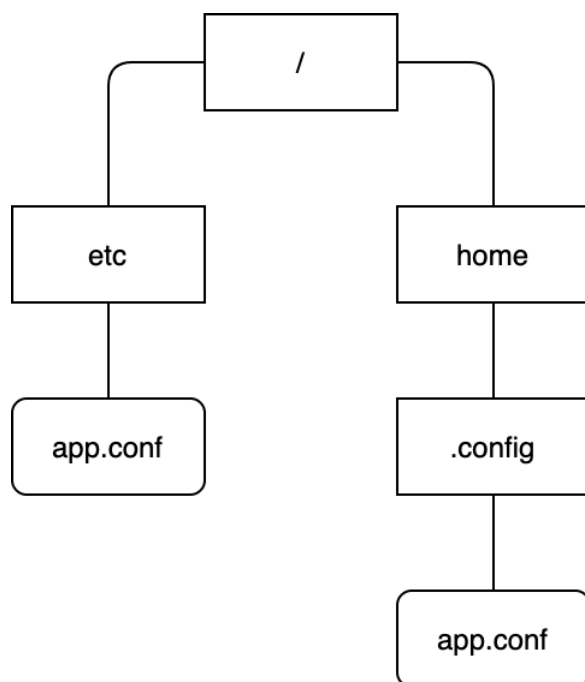


Рис. 4.2. Пример иерархии файлов

Как видно из рисунка 4.2, файловая система содержит корневой каталог /, который содержит каталоги etc и home. Каталог etc содержит конфигурационный файл app.conf. Каталог home содержит каталог .config, который содержит конфигурационный файл app.conf.

Допустим, пользователь находится в каталоге с абсолютным адресом /home и ему необходимо запустить приложение app. По

умолчанию, приложение `app` использует конфигурационный файл с абсолютным адресом `/etc/app.conf`, но у пользователя нет прав доступа для изменения данного файла и внесения в него необходимых настроек. При этом, у пользователя есть возможность вносить изменения в файлы, находящиеся в каталоге `/home/.config`. Для запуска приложения `app` с другим конфигурационным файлом необходимо передать адрес такого файла в параметрах запуска приложения. При запуске приложения `app` адрес его рабочего каталога всегда будет `/home`. Тогда, адрес нового конфигурационного файла можно передать используя не абсолютный, а относительный путь (относительно адреса рабочего каталога приложения `app`) — `.config/app.conf` или `./config/app.conf`.

При указании относительного пути также можно осуществлять перемещение по иерархии файлов и каталогов при помощи элементов `./` и `../` — текущий каталог и родительский каталог для текущего каталога соответственно. Данные элементы можно комбинировать, например, для запуска приложения `app` с конфигурационным файлом с абсолютным адресом `/etc/app.conf`, используя относительную адресацию из каталога `/home/.config`, необходимо указать следующий относительный адрес — `../../etc/app.conf`.

Таким образом, абсолютный путь необходимо использовать в случае обращения к системным файлам, имеющим редко изменяющийся стандартизированный адрес. В случае необходимости обращения к файлам, поставляемым вместе с исполняемым файлом приложения, при условии запуска такого приложения из любого каталога системы, предпочтительно использовать относительный адрес (относительно расположения исполняемого файла).

4.4 Таблица разделов

Таблица разделов определяет структуру разделов устройства хранения данных, а также путь до загрузчика операционной системы. Информация из таблицы разделов используется в процессе загрузки ОС и запуска операционной системы для получения информации о структуре разделов и их свойствах: размер раздела, тип файловой системы раздела и т. д.

В настоящее время используется два стандарта формирования таблицы разделов:

1. Master Boot Record (MBR) — главная загрузочная запись, содержит код и данные, необходимые для последующей загрузки операционной системы. Код и данные MBR расположены в первых физических секторах (чаще всего в самом первом) устройства хранения данных. MBR активно использовалась в период с 1983 по 2010 года.
2. GUID Partition Table (GPT) — стандарт формата размещения таблиц разделов на физическом устройстве хранения данных. GPT входит в состав Extensible Firmware Interface (EFI, Расширяемый микропрограммный интерфейс). EFI — стандарт, предложенный компанией Intel для замены BIOS (Basic Input/Output System, Базовая система ввода-вывода).

В процессе запуска компьютера BIOS загружает код MBR в оперативную память и передает управление загрузочному коду.

В отличие от MBR, GPT опирается на расширенные возможности EFI для осуществления процесса загрузки. При этом, GPT содержит MBR в первых физических секторах устройства хранения данных для сохранения обратной совместимости.

GPT для адресации логических блоков использует систему адресации логических блоков (Logical block addressing, LBA) вместо применяющейся в MBR адресации вида «Цилиндр-Головка-Сектор» (Cylinder, Head, Sector, CHS).

Количество разделов в GPT не ограничено стандартом и зависит от операционной системы. MBR допускает только четыре основных раздела, остальные разделы — дополнительные.

При использовании GRP для обеспечения безопасности оглавление и таблица разделов записаны как в начале, так и в конце диска.

GPT позволяет создавать разделы диска размером до 9.4 ЗБ, а MBR только до 2.2 ТБ.

4.5 Файловая система

Файловая система (file system) — способ организации, хранения и именования данных на устройствах хранения данных.

Конкретная файловая система определяет максимальную длину имен файлов и каталогов, максимальный размер файла и раздела, перечень атрибутов файла.

Файловая система позволяет организовать взаимодействие между системными вызовами для работы с файлами и каталогами и устройствами хранения данных. Для пользовательского процесса доступ к файлу осуществляется прозрачно — нет необходимости иметь сведения о способах расположения информации в файле и типе физического носителя. Доступ к файлу осуществляется по его адресу, также пользовательский процесс обладает сведениями о размере и атрибутах файла.

Для операционной системы расположение данных на устройстве хранения реализовано в виде множества физических блоков (кластеров). Размер кластера может варьироваться, например, от 4 КБ до 2 МБ.

Чем меньше кластер, тем эффективнее используется пространство устройства хранения данных (проблема внутренней фрагментации), но на медленных устройствах хранения скорость чтения большого количества кластеров минимального размера приведет к значительному увеличению времени доступа к данным и наоборот.

4.5.1 Типы файловых систем

В данном разделе будут рассмотрены некоторые типы файловых систем. Заложенные в них идеи могут стать полезными в процессе изучения особенностей работы операционных систем с данными, расположенными на устройствах хранения.

Файловые системы с журнальной структурой

Примером файловой системы с журнальной структурой является файловая система LFS (Log-structured file system, файловая система с журнальной структурой). Файловая система LFS была разработана в рамках исследований, проводимых в университете Беркли.

Основной целью файловой системы LFS было повышение скорости записи данных на жесткий диск. В основном данные на диск записываются с помощью блоков малой длины. При большом количестве таких блоков скорость записи на диск значительно снижается, так как перед записью данных необходимо выполнить позиционирование головки жесткого диска. Данные временные затраты требуются для записи каждого блока.

Файловая система LFS представляет собой журнал. Все данные ожидающие записи собираются в один непрерывный сегмент, а затем записываются в конец журнала. При этом, такой сегмент может одновременно содержать как сами данные, так и метаданные — изменения в файловой системе. Сегменты, ещё не записанные на диск, располагаются в оперативной памяти.

При таком подходе значительно усложняется поиск метаданных, связанных с определенным файлом. Также, при увеличении размера журнала, свободное пространство на устройстве может закончиться. Для освобождения свободного пространства используется очищающий поток.

В рамках работы очищающего потока происходит сканирование журнала, определение сегментов с неактуальными данными, актуальные данные такого сегмента перемещаются в оперативную память для записи в новый сегмент. Затем сегмент помечается как свободный.

Файловые системы с журнальной структурой не нашли широкого распространения, но некоторые их идеи были использованы в журналируемых файловых системах.

Журналируемые файловые системы

Основной задачей журналируемых файловых систем (Journaling file system) является ведение журнала всех операций, которые необходимо выполнить над файловой системой.

Если в операционной системе возникает сбой, при загрузке осуществляется проверка журнала файловой системы и все невыполненные операции выполняются. Данные действия позволяют повысить устойчивость файловой системы к отказам.

При журналировании все операции, записываемые в журнал, должны обладать свойством идемпотентности — возможность многократного повторения операции с получением одного и того же результата без повреждения файловой системы.

Виртуальные файловые системы

При использовании виртуальной файловой системы (Virtual file system, VFS) общая для всех файловых систем часть выносится в отдельный виртуальный уровень. Виртуальный уровень осуществляет взаимодействие с физическими файловыми системами.

ми, а все относящиеся к файлам системные вызовы направляются в виртуальную файловую систему.

4.5.2 Методы расположения информации в файлах

Методы расположения информации в файлах определяют соответствие между файлами и кластерами устройства хранения данных.

Непрерывное размещение

Непрерывное размещение является самым простым методом расположения информации в файлах. При использовании непрерывного размещения файловая система соотносит с файлом данные о начальном кластере и количестве кластеров в файле. Файл может располагаться только непрерывно.

Если на устройстве хранения данных не существует непрерывного свободного пространства необходимой длины, то файл не может быть записан (внешняя фрагментация).

При использовании непрерывного метода размещения файлов скорость чтения данных будет высокой, так как все необходимые кластеры располагаются последовательно.

На рисунке 4.3 представлен пример организации файловой системы, основанной на методе непрерывного размещения файлов.

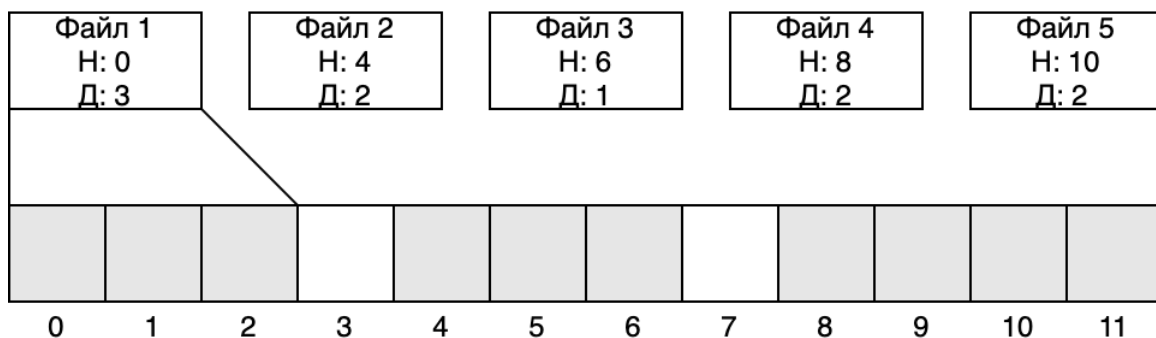


Рис. 4.3. Иллюстративный пример непрерывного размещения файлов

Размещение с использованием связанного списка

Размещение с использованием связанного списка позволяет представить каждый файл в виде связанного списка кластеров устройства хранения данных. Первое слово каждого кластера используется для хранения указателя на следующий кластер файла,

остальная часть кластера предназначена для хранения данных файла. Для получения списка кластеров, принадлежащих файлу, достаточно хранить только адрес первого элемента связанного списка.

При использовании связанного списка полностью устраняется проблема внешней фрагментации. При этом, размещение с использованием связанного списка обладает низкой скоростью доступа к данным, так как элементы связанного списка могут быть хаотично разбросаны по устройству хранения данных.

На рисунке 4.4 представлен пример организации файловой системы, основанной на методе размещения с использованием связанного списка.

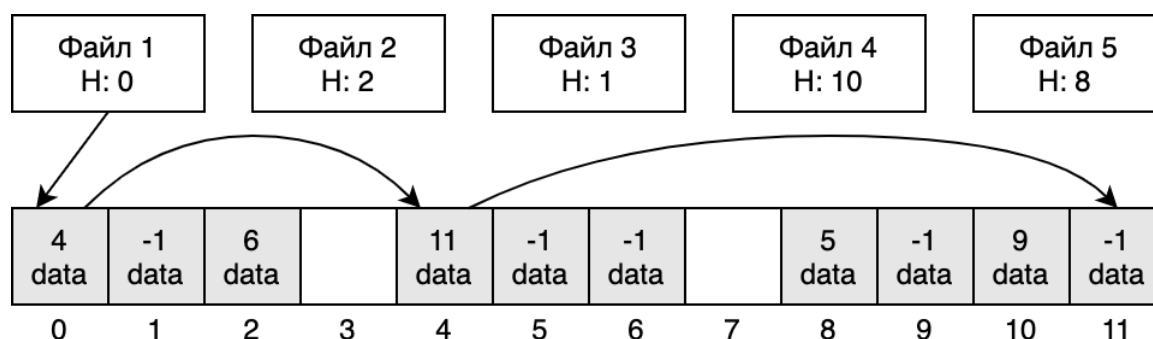


Рис. 4.4. Иллюстративный пример размещения с использованием связанного списка

Размещение с помощью связанного списка, использующего таблицу в памяти

Для повышения скорости работы связанного списка, его можно расположить в оперативной памяти, изъяв указатели из кластеров. Доступ к кластерам будет осуществляться быстрее за счет обращения к оперативной памяти, а не к устройству хранения данных.

Недостатком данного метода является использование оперативной памяти для хранения таблицы.

На рисунке 4.5 представлен пример организации файловой системы, основанной на методе размещения с помощью связанного списка, использующего таблицу в памяти.

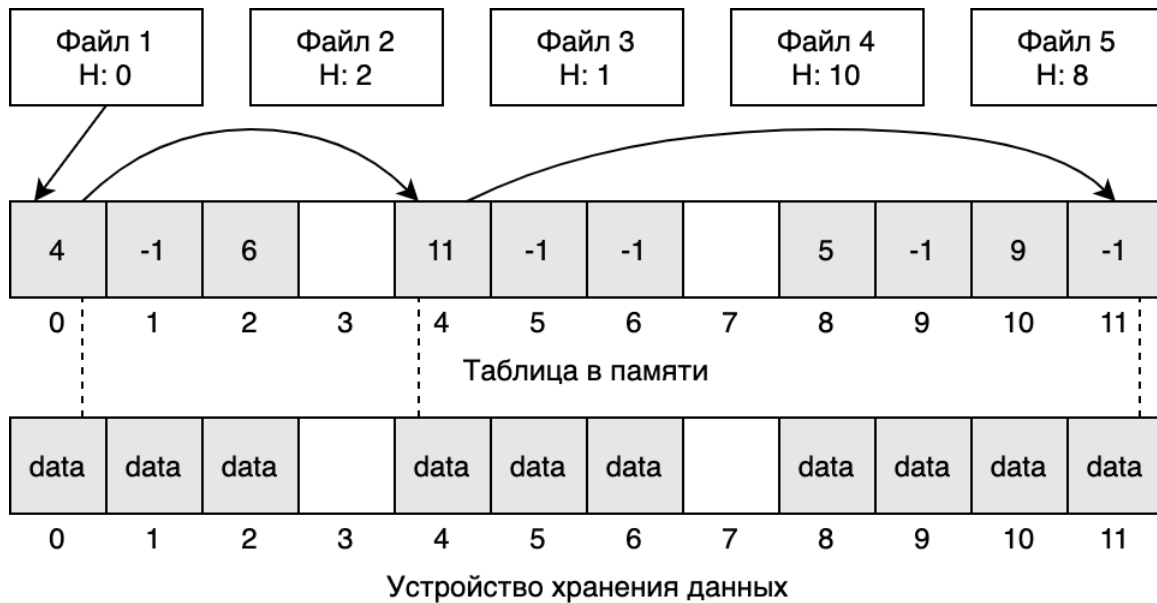


Рис. 4.5. Иллюстративный пример размещения с помощью связанного списка, использующего таблицу в памяти

i-узлы

При использовании метода размещения, основанного на *i*-узлах, каждый файл содержит ссылку на специальную структуру, содержащую информацию о кластерах файла — *i*-узел (index node, индекс-узел).

i-узлы имеют фиксированный размер и позволяют хранить информацию об атрибутах файла и его кластерах. Если размер *i*-узла достиг максимального размера, но остались кластеры, информация о которых ещё не была записана, то создается новый *i*-узел для данного файла, и в последний свободный блок текущего *i*-узла заносится ссылка на новый *i*-узел.

i-узел загружается в память только в том случае, если связанный с ним файл открыт. При этом информация о всех кластерах файла будет расположена в оперативной памяти. При использовании *i*-узлов, оперативная память будет использоваться более оптимально, так как в памяти содержатся *i*-узлы только открытых в данный момент файлов.

На рисунке 4.6 представлен пример организации файловой системы, основанной на *i*-узлах.

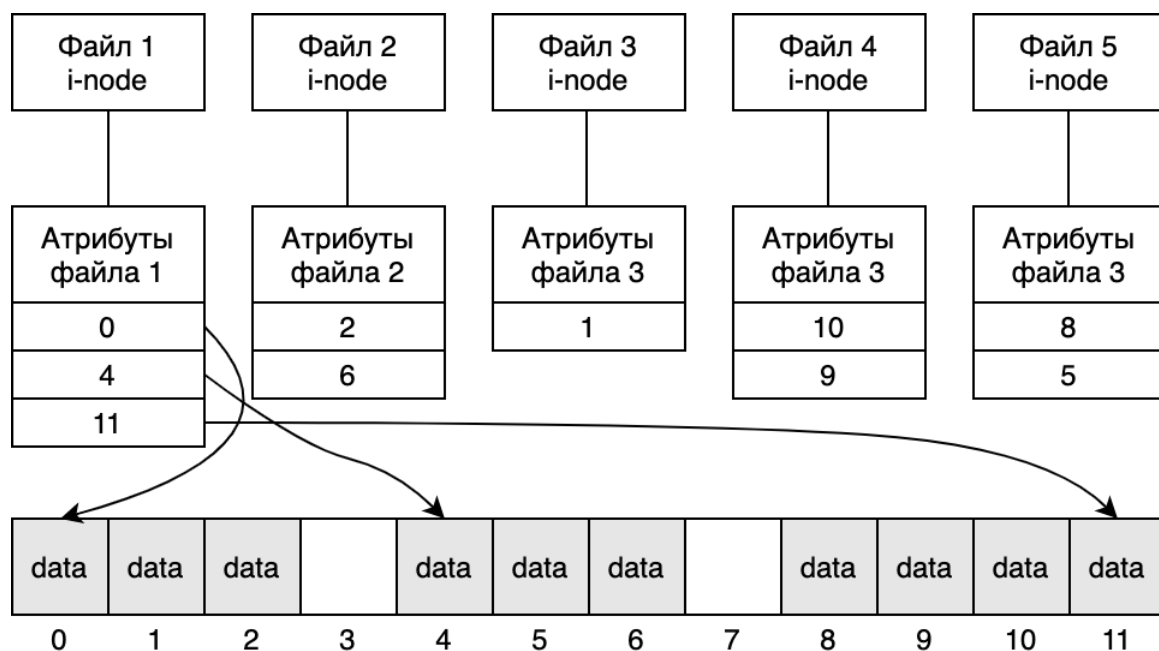


Рис. 4.6. Иллюстративный пример размещения с помощью i-узлов

4.5.3 Отслеживание свободных кластеров

Для отслеживания информации о свободных кластерах обычно используются следующие структуры данных:

- **связанный список.** При использовании связанного списка, каждый элемент списка содержит такое множество номеров свободных кластеров, сколько в него может поместиться. Например, при кластерах размером 1 Кбайт с 32-разрядным номером, каждый элемент связанного списка может вместить номера 255 кластеров. Как правило, для хранения списка свободных кластеров используются свободные кластеры.
- **битовый массив.** При использовании битового массива, для хранения сведений о свободных кластерах диска, имеющего n кластеров, необходим битовый массив длиной n . Битовый массив требует меньше свободного пространства устройства хранения, поскольку для хранения сведений о занятости кластера в нем используется один бит. Для хранения связанного списка требуется меньше дисковой памяти, чем для битового массива, только если диск почти заполнен.

На рисунке 4.7 представлен пример хранения сведений о свободных кластерах с применением связанного списка (рис. 4.7.а) и битового массива (рис. 4.7.б).

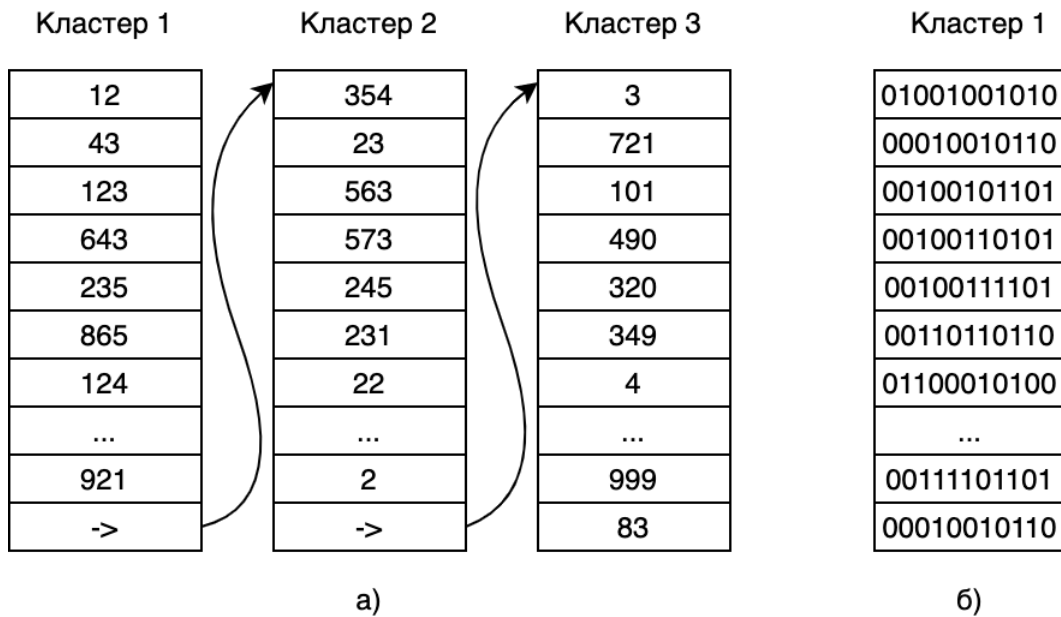


Рис. 4.7. Иллюстративный пример применения связанного списка и битового массива для хранения сведений о свободных кластерах

4.5.4 Реализация каталогов

Для получения доступа к файлу, необходимо найти ссылку на данный файл в соответствующем каталоге. Доступ к записи о файле необходим для получения сведений о кластерах файла и его атрибутах. В зависимости от метода размещения эта информация может быть дисковым адресом всего файла (для непрерывного размещения), номером первого блока (для методов, основанных на связанных списках) или номером *i*-узла.

Для хранения атрибутов файлов обычно используются следующие подходы:

- хранение атрибутов в записи каталога фиксированной длины — в данном случае под запись элемента каталога (имя и атрибуты) отводится запись фиксированной длины. Данный метод достаточно легко реализовать, но фиксированная длина записи накладывает ограничения на длину имени элемента;
- хранение атрибутов в записи каталога с переменной длиной — в данном случае под запись элемента каталога (атрибуты) отводится запись фиксированной длины, а имя элемента хранится в отдельной области памяти. Данный метод позволяет задавать длинные имена для элементов каталога, но управле-

ние отдельной областью памяти требует решения различных проблем.

Для поиска элемента каталога по его имени можно использовать обычный линейный поиск, но при увеличении количества элементов, данный метод будет иметь низкую скорость работы.

Для ускорения поиска элемента каталога по имени обычно используются хэш-таблицы или кэширование. Данные методы значительно ускоряют скорость поиска, но сложны в реализации и требуют дополнительной памяти для хранения данных.

4.5.5 Непротиворечивость файловой системы

Противоречивость файловой системы — проблема, при которой сведения о свободных кластерах не соответствует сведениями о занятых кластерах. Многие файловые системы считывают кластеры, вносят в них изменения, а потом записывают их обратно на устройство хранения данных. Если при сбое в работе операционной системы на устройство хранения записаны не все модифицированные кластеры, файловая система может оказаться в противоречивом состоянии.

В процессе проверки непротиворечивости сведений о занятых и свободных кластерах создаются две таблицы:

- для хранения сведений о занятых кластерах,
- для хранения сведений о свободных кластерах.

Для формирования таких таблиц используется анализ метаданных файловой системы.

На рисунке 4.8 представлен пример состояний файловой системы. Как видно из рисунка 4.8.а, если сведения о свободных кластерах соответствуют сведениям о занятых кластерах — для каждого кластера счетчик в одной таблице содержит значение 0, а в другой 1 или наоборот — файловая система считается непротиворечивой.

В некоторых случаях сведения о кластерах могут отсутствовать в обеих таблицах. Например, на рисунке 4.8.б у кластера с номером 2 оба счетчика имеют значение 0. Такой кластер является пропавшим. Хотя пропавшие кластеры не влияют на работоспособность файловой системы, наличие пропавших кластеров снижает емкость устройства хранения. Для исправления пропав-

шего кластера необходимо добавить сведения о нем в список или массив свободных кластеров.

Занято	1	1	0	1	0	1
	0	1	2	3	4	5
Свободно	0	0	1	0	1	0

а)

Занято	1	1	0	1	0	1
	0	1	2	3	4	5
Свободно	0	0	0	0	1	0

б)

Занято	1	1	0	1	0	1
	0	1	2	3	4	5
Свободно	0	0	1	0	2	0

в)

Занято	1	1	0	1	0	2
	0	1	2	3	4	5
Свободно	0	0	1	0	1	0

г)

Рис. 4.8. Состояния файловой системы

Иногда кластеры могут дважды фигурировать в таблице свободных кластеров, например, на рисунке 4.8.в таким кластером является кластер с номером 4. Кластер может быть два раза отмечен как незанятый только при использовании связанного списка для отслеживания свободных кластеров. Для устране-

ния подобной проблемы требуется перестроить список свободных кластеров.

На рисунке 4.8.в представлена ситуация, когда кластер одновременно связан с двумя различными файлами, например кластер с номером 5. При удалении одного из файлов, такой кластер будет помечен как свободный и занятый одновременно, что может привести к будущим проблемам чтения другого файла, который также ссылается на данный кластер. Если будут удалены оба файла, то блок попадет в список свободных блоков дважды.

Для устранения данной проблемы необходимо выделить свободный кластер, скопировать в него содержимое проблемного кластера, а затем в одном из файлов заменить ссылку с проблемного кластера на новый.

Подобная проверка может осуществляться не на уровне кластеров, а на уровне файлов, для проверки корректности метаданных файловой системы.

4.6 Контрольные вопросы

1. Что такое файл?
2. Что такое каталог?
3. В чем различия между файлом и каталогом?
4. Какие структуры данных используются для хранения информации о свободных кластерах?
5. Какие методы используются для расположения информации в файлах?
6. Что такое непротиворечивость файловой системы?
7. Что такое абсолютная адресация?
8. Что такое относительная адресация?
9. Какие виды ссылок на файлы существуют?

5 ВВОД И ВЫВОД ИНФОРМАЦИИ

В настоящее время существует огромное количество разновидностей и моделей устройств ввода-вывода, но условно их можно разделить на две категории:

1. Блочные устройства — устройства, хранящие информацию в блоках фиксированной длины, у каждого из которых есть свой собственный адрес. Передача данных осуществляется через пакеты, состоящие из одного или нескольких последовательных блоков. Блочное устройство позволяет читать или записывать каждый блок независимо от остальных блоков. К блочным устройствам можно отнести твердотельные накопители, магнитные диски, флэш-накопители, приводы для компакт-дисков и т. д.
2. Символьные устройства — устройства принимающие или порождающие поток символов, не имеющий блочной структуры. Символьные устройства не содержат адресов и следовательно не позволяют выполнять операцию позиционирования. Примером символьных устройств являются принтеры, сетевые интерфейсы, клавиатуры и т. д.

Некоторые устройства ввода-вывода не попадают под данную классификацию. Например, часы генерируют прерывания через определенный промежуток времени и не являются символьным или блочным устройством.

Устройства ввода-вывода могут быть встроены в системную плату, подключаться к различным внутренним и внешним шинам системной платы. Устройства ввода-вывода, работающие с ВС через внешние разъемы системной платы, могут быть подключены или отключены в процессе работы ВС. К устройствам ввода-вывода могут одновременно обращаться различные пользовательские и системные процессы. Вероятность возникновения различного рода ошибок при работе с устройствами ввода-вывода является достаточно высокой. Решить проблемы и скрыть сложность работы с устройствами ввода-вывода позволяет подсистема управления устройствами ввода-вывода операционной системы, являющаяся самой значительной частью операционной системы.

С точки зрения подсистемы управления устройствами ввода-вывода любое устройство ввода-вывода состоит из аппаратной со-

ставляющей — устройство ввода-вывода, и программной составляющей — драйвер устройства.

5.1 Аппаратное обеспечение ввода-вывода

Аппаратное обеспечение ввода-вывода включает в себя электронную составляющую — контроллер устройства ввода-вывода и «механическую» составляющую — остальная часть устройства ввода-вывода.

5.1.1 Контроллер устройства

Контроллер устройства ввода-вывода позволяет операционной системе взаимодействовать с устройством ввода-вывода. Контроллер содержит некоторое множество регистров. Запись определенных значений в регистры контроллера позволяет управлять поведением «механической» составляющей устройства ввода-вывода: получение данных, запись данных, включение или отключение устройства и т. д. Чтение данных регистров контроллера позволяет определить состояние устройства ввода-вывода.

Контроллер позволяет скрыть низкоуровневые операции с устройством ввода-вывода, например, подготовка данных, обработка ошибок работы с устройством ввода-вывода, запись данных в оперативную память, получение данных из оперативной памяти и т. д.

В дополнение к регистрам контроллера, многие устройства ввода-вывода имеют буфер данных. Буфер данных используется операционной системой для считывания и записи данных.

5.1.2 Работа с контроллером

Для работы с регистрами контроллера и буфером данных устройства ввода-вывода осуществляется их отображение на пространство памяти. Такое отображение может быть реализовано с помощью следующих подходов:

1. Каждому регистру контроллера назначается номер порта ввода-вывода. Набор всех портов ввода-вывода формирует пространство портов ввода-вывода, которое защищено от доступа со стороны пользовательских процессов. Для взаимодействия с портами ввода-вывода используются специальные команды ввода-вывода. Например для чтения данных

из регистра контроллера может использоваться команда `IN REG, PORT`, а для записи данных в регистр контроллера команда `OUT PORT, REG`.

2. Отображение всех регистров контроллера на пространство оперативной памяти — каждому регистру контроллера присваивается уникальный адрес в оперативной памяти, который не распределяется между пользовательскими процессами. Обычно регистры контроллера отображаются на адреса, расположенные в верхней части адресного пространства.
3. Гибридный подход.

При необходимости получения значения регистра контроллера, отображенного на память или представленного портом ввода-вывода, адрес данного регистра контроллера (или порта ввода-вывода) выставляется на адресных линиях шины, а затем выставляется сигнал линии управления шиной на чтение данных. Для определения пространства памяти — ввода-вывода или памяти — используется специальная сигнальная линия шины: если необходимо обратиться к пространству памяти, то на запрос отвечает память, если необходимо обратиться к пространству ввода-вывода, то на запрос отвечает устройство ввода-вывода.

При отображении регистров контроллера на пространство памяти для обеспечения безопасности производится сравнение адреса, выставленного на адресной линии шины, с диапазоном адресов обслуживаемых устройством ввода-вывода. Если адрес попадает в обслуживаемый устройством диапазон, то устройство ввода-вывода отвечает на запрос. Диапазоны адресов, выделенные устройствам ввода-вывода, не выделяются пользовательским процессам.

Преимущества и недостатки использования портов ввода-вывода:

1. Преимущества:
 - отсутствие проблемы кэширования страниц оперативной памяти.
 - отсутствие необходимости поиска устройства ввода-вывода, которому предназначены данные.
2. Недостатки:
 - для взаимодействия с портами ввода-вывода необходимо использовать язык ассемблера.

- необходим механизм защиты портов ввода-вывода от воздействия пользовательских процессов.
- требуются дополнительные затраты процессорного времени для помещения данных регистров контроллера в оперативную память.

Преимущества и недостатки отображения регистров контроллера на пространство оперативной памяти:

1. Преимущества:

- регистры контроллера, отображенные на пространство памяти, являются обычными переменными. Обратиться к регистрам контроллера можно с помощью команд на языках высокого уровня.
- для работы с регистрами контроллера, отображенными на пространство памяти, требуется меньше затрат процессорного времени.
- отсутствие необходимости защиты регистров контроллера от воздействия пользовательских процессов.

2. Недостатки:

- необходима поддержка механизма запрета кэширования отдельных страниц оперативной памяти.
- необходим механизм определения устройства ввода-вывода, для контроллера которого предназначены записанные в память данные.

5.2 Прерывания

Если взаимодействие с устройством ввода-вывода осуществляется без использования прерываний, пользовательский процесс выполняет некоторый системный вызов и блокируется, происходит переключение в пространство ядра, а ядро операционной системы взаимодействует с драйвером данного устройства. Драйвер опрашивает устройство и ожидает окончания операции ввода-вывода. После окончания операции ввода-вывода управление возвращается операционной системе, а затем и пользовательскому процессу. Данный режим работы с устройством ввода-вывода называется активным ожиданием и имеет существенный недостаток — процессорное время тратится на опрос устройства ввода-вывода.

При использовании прерываний драйвер устройства инициирует работу устройства ввода-вывода и возвращает управление операционной системе. Пользовательский процесс остается в состоянии блокировки, а вместо него выполняются другие пользовательские процессы. После окончания операции ввода-вывода контроллер устройства генерирует прерывание, выполнение текущего процесса прерывается, активируется драйвер устройства, а управление возвращается операционной системе, которая переводит заблокированный пользовательский процесс в состояние готовности. Для обработки прерываний используется контроллер прерываний центрального процессора. Прерывания используются для более оптимального использования процессорного времени при работе с устройствами ввода-вывода.

Для обработки прерывания контроллер устройства ввода-вывода помещает идентификатор на адресные линии, указывая, какое устройство требует к себе внимания, и выставляет сигнал на прерывание работы центрального процессора. Сигнал на прерывание приводит к тому, что центральный процессор прерывает работу над текущим процессом и приступает к обработке прерывания.

5.2.1 Вектор прерываний

Идентификатор установленный контроллером устройства ввода-вывода на адресных линиях используется в качестве индекса в таблице, называемой вектором прерываний. Каждая строка вектора прерывания содержит значение счетчика команд.

Счетчик команд указывает на адрес процедуры обработки прерывания. Процедура обработки прерывания подтверждает начало обработки прерывания, записывая определенное значение в один из портов ввода-вывода контроллера прерываний. После этого процессор готов обрабатывать новые прерывания. Данная процедура используется для устранения конкуренции при обработке нескольких прерываний.

Все источники прерываний делятся на классы и каждому классу назначается свой уровень приоритета запроса на прерывание. Существует две стратегии обработки прерываний при использовании приоритизации:

1. Относительное обслуживание прерываний — если во время обработки прерывания поступает более приоритетное прерывание, то это прерывание будет обработано только после завершения текущей процедуры обработки прерывания.
2. Абсолютное обслуживание прерываний — если во время обработки прерывания поступает более приоритетное прерывание, то текущая процедура обработки прерывания вытесняется, и процессор начинает выполнять обработку вновь поступившего более приоритетного прерывания. После завершения этой процедуры процессор возвращается к выполнению вытесненной процедуры обработки прерывания.

5.2.2 Виды прерываний

В зависимости от источника возникновения прерывания делятся на:

- асинхронные, или внешние (аппаратные) — исходят от устройств ввода-вывода и могут произойти в любой произвольный момент времени. Устройство ввода-вывода генерирует прерывания для обработки выполненных устройством команд операционной системой;
- синхронные, или внутренние — события, происходящие при обработке команд в центральном процессоре: деление на ноль или переполнение стека, обращение к недопустимым адресам памяти или недопустимый код операции;
- программные (частный случай внутреннего прерывания) — исполнением специальной инструкции в коде программы.

5.2.3 Осуществление операций ввода-вывода

Существует несколько способов обеспечения операций ввода-вывода.

Программный ввод-вывод

При использовании программного ввода-вывода пользовательский процесс генерирует программные прерывания для взаимодействия с устройством ввода-вывода. Например, для отправки на устройство ввода-вывода пяти текстовых символов выполняется следующая последовательность шагов:

1. Обращение к системному вызову для отправки символов в устройство ввода-вывода.
2. Блокировка пользовательского процесса, переход в пространство ядра.
3. Получение данных из буфера.
4. Обращение к драйверу устройства для проверки доступности устройства ввода-вывода. Если устройство ввода-вывода занято, операционная система ожидает освобождения устройства.
5. Вызов функции драйвера для отправки символа в устройство ввода-вывода.
6. Ожидание окончания операции.
7. Если остались необработанные символы, перейти к шагу 3, иначе перейти к шагу 8.
8. Возврат управления пользовательскому процессу

Программный ввод-вывод прост в реализации. При использовании программного ввода-вывода центральный процессор занят ожиданием завершения операции ввода-вывода.

Ввод-вывод, управляемый прерываниями

В отличие от программного ввода-вывода, при использовании ввода-вывода, управляемого прерываниями, операционная система не ожидает завершения операции ввода-вывода, а выполняет функцию планирования и запускает следующий доступный процесс. В данном случае выполняется следующая последовательность шагов:

1. Обращение к системному вызову для отправки символов в устройство ввода-вывода.
2. Блокировка пользовательского процесса, переход в пространство ядра.
3. Получение данных из буфера.
4. Обращение к драйверу устройства для проверки доступности устройства ввода-вывода. Если устройство ввода-вывода занято, операционная система ожидает освобождения устройства.
5. Вызов функции драйвера для отправки символа в устройство ввода-вывода. После окончания данной операции будет сгенерировано асинхронное прерывание (переход к шагу 8).

6. Возвращение управления операционной системе.
7. Планирование процессов. Запуск доступного процесса.
8. Если остались необработанные символы, перейти к шагу 3, иначе перейти к шагу 9.
9. Перевод пользовательского процесса в состояние готовности.

Недостатком ввода-вывода, управляемого с помощью прерываний, является генерация прерывания для каждого символа, на обработку которого требуется некоторое процессорное время.

Ввод-вывод с использованием DMA

DMA (Direct Memory Access) — прямой доступ к памяти.

Для применения ввода-вывода с использованием DMA необходимо наличие в ВС аппаратного DMA-контроллера. DMA-контроллер может располагаться на системной плате или в контроллере устройства ввода-вывода.

При использовании DMA центральный процессор программирует DMA-контроллер, устанавливая значения его регистров для работы с контроллером устройства ввода-вывода. После завершения работы с устройством ввода-вывода DMA-контроллер генерирует прерывание и пользовательский процесс, ожидающий выполнение операции, переводится в состояние готовности.

Фактически, ввод-вывод с использованием DMA работает аналогично вводу-выводу, управляемому с помощью прерываний, однако вместо центрального процессора прерывания от устройства ввода-вывода обрабатывает DMA-контроллер.

Таким образом, количества прерываний, которые необходимо обработать центральному процессору, сокращается с одного на каждый символ до одного.

DMA-контроллеры обычно работают намного медленнее, чем центральный процессор.

5.3 Программное обеспечение ввода-вывода

При разработке подсистемы управления устройствами ввода-вывода операционной системы обычно следуют принципу независимости от конкретных устройств. При котором все специфичные для определенного устройства ввода-вывода (или группы устройств) особенности должны быть реализованы на

более низком уровне. Таким образом, появляется возможность создания программ, способных получить доступ к любому устройству ввода-вывода без необходимости предварительного определения устройства.

5.3.1 Задачи программного обеспечения ввода-вывода

Программное обеспечение ввода-вывода обычно направлено на решение следующих задач:

- однообразное наименование устройств ввода-вывода — имя устройства не должно зависеть от его типа;
- обработка ошибок — обработка ошибок должна осуществляться на наиболее низком из возможных уровней;
- определения способа передачи данных — представление для пользовательских процессов асинхронных операций с устройством ввода-вывода в виде синхронных операций для облегчения разработки и отладки программного обеспечения;
- буферизация — возможность накапливать данные в буфере данных устройства ввода-вывода для дальнейшей обработки или отправки.

5.3.2 Уровни программного обеспечения ввода-вывода

Программное обеспечение ввода-вывода представляет собой иерархию уровней над аппаратным обеспечением (рис. 5.1).

Каждый уровень программного обеспечения ввода-вывода имеет свой интерфейс и предоставляет определенные функции уровню, расположенному выше.

Обработчик прерываний

Обработчик прерываний обрабатывает прерывания, поступившие от устройств ввода-вывода, для активации драйвера устройства ввода-вывода и обработки результата операции с устройством.

Стандартная библиотека языка программирования
Подсистема управления устройствами ввода-вывода
Слой аппаратный абстракций
Драйверы
Обработчики прерываний
Аппаратное обеспечение

Рис. 5.1. Уровни программного обеспечения ввода-вывода

Драйверы

Драйверы устройств ввода-вывода используются для взаимодействия с контроллерами устройства. Контроллер устройства ввода-вывода содержит набор регистров, через взаимодействие с которыми осуществляется управление устройством. Состав и принцип работы с регистрами контроллера могут сильно различаться даже в рамках одного семейства устройств. Для сокрытия сложности работы с контроллером устройства ввода-вывода и абстрагирования от различий контроллеров используются драйверы. Драйвер представляет собой программное обеспечение, используемое операционной системой для взаимодействия с устройством ввода-вывода.

В большинстве случаев драйвер устройства ввода-вывода создается производителем данного устройства и учитывает все особенности работы с контроллером. Для различных операционных систем необходима разработка отдельных версий драйверов.

Обычно драйвер управляет определенным типом устройств ввода-вывода, но в некоторых случаях драйвер может управлять целым классом родственных устройств.

Драйвер устройства может содержаться в ядре операционной системы (UNIX), работать в пространстве ядра в виде динамически подключаемого к ядру операционной системы модуля

(Linux, Windows NT) либо исполняться в пространстве пользователя (Minix).

При использовании драйверов устройств ввода-вывода необходимо стандартизировать интерфейс для взаимодействия с драйвером со стороны операционной системы. Подобный стандартизированный интерфейс драйвера позволяет обеспечить неизменность уровней программного обеспечения ввода-вывода, расположенных выше. Обычно выделяют два типа стандартизированных интерфейсов:

- для блочных устройств ввода-вывода,
- для символьных устройств ввода-вывода.

Фактически, интерфейс драйвера определяет перечень необходимых для реализации процедур, которые может использовать операционная система, а сам драйвер является реализацией данных процедур.

К основным функциям драйвера устройства ввода-вывода можно отнести:

1. Выполнение абстрактных запросов на запись и/или чтение данных устройства ввода-вывода путем их преобразования в команды контроллера устройства с учетом его особенностей.
2. Отслеживание порядка выполнения операций ввода-вывода.
3. Инициализация устройства.
4. Регистрация событий ввода-вывода.
5. Управление энергопотреблением устройства.

Общий принцип работы драйверов устройств ввода-вывода состоит из следующих шагов:

1. Проверка корректности входных данных.
2. Проверка доступности устройства ввода-вывода.
3. Формирование очереди обработки команд.
4. Инициализация устройства ввода-вывода для начала работы с ним.
5. Взаимодействие с контроллером устройства ввода-вывода (трансляция абстрактных команд в команды контроллера устройства).
6. Блокировка драйвера устройства до окончания выполнения команды.
7. Проверка результата исполнения команд контроллера.
8. Возврат результата выполнения команды.

Данные шаги описывают принцип работы драйвера устройства ввода-вывода на достаточно простом для понимания уровне.

Слой аппаратных абстракций

Слой аппаратных абстракций (Hardware Abstraction Layer, HAL) предназначен для скрытия различий в аппаратном обеспечении от подсистемы управления устройствами ввода-вывода для абстрагирования большей части ядра операционной системы от особенностей работы с аппаратным обеспечением.

Подсистема управления устройствами ввода-вывода

Подсистема управления устройствами ввода-вывода используется для реализации следующих функций работы с устройствами ввода-вывода, не зависящих от аппаратного обеспечения:

1. Предоставление унифицированного интерфейса для драйверов устройств. Для каждого класса устройств операционная система определяет набор функций, которые должен реализовывать драйвер. При загрузке драйвера операционная система записывает адрес таблицы указателей на функции драйвера. Таблица указателей на функции драйвера определяет интерфейс между драйвером и остальной частью ядра операционной системы. Все устройства определенного класса должны соответствовать одному интерфейсу.
2. Присвоение имен устройствам ввода-вывода для определения соответствия между устройством и его драйвером. Устройства ввода-вывода представляются в файловой системе в виде именованных объектов.
3. Буферизация. Буферизация используется для передачи данных из пользовательского процесса в драйвер устройства ввода-вывода большими порциями без необходимости обработки каждого отдельного символа, пакета и т. д. Буферизация используется для сокращения времени ожидания окончания операции ввода-вывода пользовательским процессом. Чрезмерное использование буферизации может привести к снижению производительности ВС.
4. Обработка ошибок и вывод сообщений об ошибках.
5. Распределение и освобождение устройств ввода-вывода. Данная функция используется для предоставления моно-

польного доступа к устройству ввода-вывода со стороны пользовательских процессов. Обычно для этих целей используется отображение устройства на файловую систему и применение системных вызовов для работы с файлами open и close. В зависимости от реализации данной функции, при обращении пользовательского процесса к занятому устройству ввода-вывода возвращается ошибка доступа либо пользовательский процесс блокируется и ожидает освобождения устройства.

Стандартная библиотека языка программирования

Определенные функции и процедуры стандартной библиотеки языка программирования используются для абстрагирования от особенностей работы с системными вызовами операционной системы для взаимодействия с подсистемой управления устройствами ввода-вывода. Такие процедуры и функции позволяют корректно вызывать системные вызовы подсистемы управления устройствами ввода-вывода и передавать необходимые для их работы параметры. Стандартная библиотека языка программирования позволяет облегчить разработку прикладных программ.

5.4 Контрольные вопросы

1. Для чего используется контроллер устройства?
2. Для чего используется драйвер устройства?
3. Для чего используется программное обеспечение ввода-вывода?
4. Каким образом операционная система взаимодействует с устройством ввода-вывода?
5. Для чего используются прерывания?
6. Какие виды прерываний существуют?
7. Как устроен и для чего используется DMA-контроллер?

6 СОСТЯЗАТЕЛЬНЫЕ И ТУПИКОВЫЕ СИТУАЦИИ

Вычислительная система представляет собой набор ресурсов: области оперативной памяти, устройства ввода-вывода, элементы файловой системы и т. д. С такими ресурсами в одну единицу времени может взаимодействовать множество процессов. В некоторых случаях может возникнуть состязательная ситуация, при которой несколько процессов работают с одним ресурсом. В таком случае данные, содержащиеся в таком ресурсе, могут быть повреждены, искажены или потеряны. Для решения проблемы возникновения состязательных ситуаций во многих операционных системах используются различные механизмы блокировки для предоставления исключительного доступа к ресурсу только для одного процесса.

Однако процессам одновременно может понадобиться доступ к нескольким ресурсам, в таком случае решение проблемы возникновения состязательных ситуаций может привести к возникновению тупиковой ситуации.

В данной главе будут рассмотрены некоторые механизмы решения проблемы возникновения состязательных ситуаций, а также методы обнаружения и предотвращения возникновения тупиковых ситуаций.

6.1 Состязательные ситуации

Состязательная ситуация – ситуация, при которой два или более процесса/потока считывают или записывают какие-либо общие данные, а окончательный результат зависит от порядка выполнения процессов/потоков. Часть программы, в которой происходит обращение к общим ресурсам называется критической областью.

Для примера рассмотрим следующий программный код на языке Java:

```
public class KnowledgeBaseApplication {
    private static int count = 0;

    private static void calc() {
        for (int i = 0; i < 5000; i++) {
            count++;
        }
    }
}
```



```
public static void main(String[] args)
    throws InterruptedException {
    Thread t1 = new Thread(() -> calc());
    Thread t2 = new Thread(() -> calc());
    t1.start(); t2.start();
    t1.join(); t2.join();
    System.out.println(count);
}
}
```

Переменная `count` является критической областью, так как потоки `t1` и `t2` обращаются к ней как к общему ресурсу. После выполнения данной программы результат должен быть равен 10000. Однако после нескольких запусков результаты работы программы выглядят следующим образом:

1. 5949
2. 10000
3. 7358
4. 6857
5. 7455

Подобные результаты вызваны следующими причинами:

- работа цикла, представленного в методе `calc()`, может быть прервана в любой момент времени из-за окончания кванта времени, выделенного текущему потоку;
- оператор инкремента `count++` состоит из трех операций: чтение значения переменной `count` – помещение значения в регистр центрального процессора, увеличение значения переменной на единицу с сохранением результата в регистр центрального процесса, запись содержимого регистра в область памяти, связанную с переменной `count`;
- если прерывание работы первого потока произошло до момента записи результата суммирования в память, то второй поток начинает работу со старым значением переменной `count`. После возврата к выполнению первого потока происходит процесс восстановления контекста исполнения и значение переменной `count` заменяется значением регистра после выполнения прерванной операции инкремента `count++`. Таким образом, результат работы второго потока замещается результатом работы первого потока – значение переменной `count` уменьшается.

Для предотвращения одновременного входа нескольких процессов/потоков в критическую область используются различные механизмы.

6.1.1 Механизмы решения проблемы возникновения состязательных ситуаций

Запрещение прерываний

Если процесс/поток входит в критическую область, то такой процесс/поток не должен быть прерван до момента выхода из критической области. Запрет прерываний может привести к проблемам в работе системы при возникновении ошибок или бесконечных циклов в исполняемом процессе/потоке.

Блокирующие переменные

В коде программы создается переменная-флаг. Для входа потока в критическую область необходимо, чтобы значение флага было сброшено. Поток устанавливает значение такой переменной и входит в критическую область. После завершения работы в критической области значение флага сбрасывается. Однако, в некоторых случаях, блокирующая переменная сама может стать критической областью.

Строгое чередование

При строгом чередовании поток 1 может войти в критическую область, если значение блокирующей переменной равно 0, а поток 2, если значение блокирующей переменной равно 1. После выхода из критической области потока 1 блокирующая переменная устанавливается в 1, а после выхода из критической области потока 2 значение устанавливается в 0. В данном случае, при входе в критическую область потоки могут находиться в режиме активного ожидания, что замедляет работу системы.

Алгоритм Петерсона

Работа алгоритма Петерсона основана на использовании переменной и массива, размер которого равен количеству потоков – каждому потоку соответствует собственный элемент массива. Поток при входе в критическую область заносит свой идентификатор в переменную и ожидает, пока элементы массива остальных пото-

ков не будут равно 0, после этого поток устанавливает в своем элементе массива значение 1 и входит в критическую область. После выхода из критической области значение соответствующего элемента массива устанавливается в 0. Переменная в данном случае позволяет определить очередность входа потоков в критическую область.

Команда TSL

TSL (Test and Set Lock, проверить и установить блокировку) – аппаратная реализация механизма блокирующих переменных в виде специального регистра центрального процессора, гарантирующая атомарность (непрерывность) операций чтения и записи значений регистра.

Семафоры

Семафор – специальный тип данных, гарантирующий атомарность операций чтения и записи значений. Семафоры позволяют увеличивать или уменьшать свое значение. Если значение семафора равно 0, то потоки будут ожидать доступа к критической области, пока значение семафора не будет иметь значение 1 или выше.

Мьютексы

Мьютекс – упрощенная реализация семафора, не предполагающая использования счетчика.

Мониторы

Монитор – специальный механизм, встроенный в язык программирования, позволяющий синхронизировать доступ к критическим областям на уровне операторов языка программирования.

Передача сообщений

Передача сообщений позволяет регулировать доступ к критической области со стороны нескольких процессов. Например, для входа в критическую область процесс может разослать сообщения остальным процессам и запросить у них доступ к критической области. Если остальные процессы не находятся в критической области, то они отправляют ответное сообщение и разреша-

ют процессу войти в критическую область. Процесс может войти в критическую область только в том случае, если все процессы прислали ответные сообщения.

Барьеры

Барьеры позволяют синхронизировать работу группы процессов. Например, некоторый алгоритм делится на части, разделенные барьерами. Процесс не может перейти через барьер, пока остальные процессы не закончат свою работу в текущей части алгоритма.

6.2 Тупиковые ситуации

Тупиковая ситуация в группе процессов возникает в том случае, если каждый процесс из этой группы ожидает события, наступление которого зависит исключительно от другого процесса из этой же группы.

Тупиковая ситуация возникает, например, когда два процесса Процесс1 и Процесс2 пытаются использовать ресурсы Ресурс1 и Ресурс2. При этом Процесс1 заблокировал Ресурс1 и ему требуется Ресурс2 для продолжения работы, но Ресурс2 заблокирован процессом Процесс2 и для освобождения ресурса Ресурс2 процессу Процесс2 требуется доступ к ресурсу Ресурс1. Возникает циклическая зависимость, приводящая к возникновению взаимоблокировки процессов.

Обычно выделяют два типа ресурсов:

1. Выгружаемый ресурс – ресурс, который безболезненно можно забрать у процесса, например, память.
2. Невыгружаемый ресурс – ресурс, который нельзя забрать у процесса без потери данных, например, принтер.

Тупиковая ситуация возникает только при борьбе за невыгружаемые ресурсы. Тупиковая ситуация предполагает наличие как минимум двух процессов и двух ресурсов.

Также выделяют следующие условия возникновения тупиковой ситуации:

1. Условие взаимного исключения – в какой-то момент времени, ресурс занят только одним процессом или свободен.
2. Условие удержания и ожидания – процесс, удерживающий ресурс, может запрашивать новые ресурсы.

3. Условие отсутствия принудительной выгрузки ресурса.
4. Условие циклического ожидания – должна существовать круговая последовательность из процессов, в которой каждый процесс ожидает доступа к ресурсу, удерживаемому следующим членом последовательности.

Только при выполнении всех условий может произойти взаимоблокировка процессов.

6.2.1 Методы борьбы с взаимоблокировками

Игнорирование проблемы

Игнорирование проблемы (страусиный алгоритм) – если вероятность взаимоблокировки достаточно мала, то решать такую проблему может быть нецелесообразно, так как добавление такой функциональности может усложнить операционную систему и привести к возникновению ошибок. Также не существует методов, которые гарантированно обнаружат возникновение взаимоблокировки. Страусиный алгоритм используется во многих современных операционных системах, так как остальные методы достаточно сложно реализовать.

Обнаружение и восстановление

При использовании данного метода операционная система пытается обнаружить взаимоблокировку и устранить ее.

Для обнаружения взаимоблокировки может быть использован следующий алгоритм. Введем некоторые обозначения:

- вектор существующих ресурсов $E = (E_1, E_2, \dots, E_m)$, содержащий m типов ресурсов, например, принтер, сканер и т. д.;
- вектор доступных ресурсов $A = (A_1, A_2, \dots, A_m)$;
- матрица текущего распределения размером $m \times n$, в которой обозначены процессы в количестве n и используемые ими ресурсы (заблокированные) каждого типа m :

$$C = \begin{pmatrix} C_{11} & C_{12} & \dots & C_{1m} \\ C_{21} & C_{22} & \dots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \dots & C_{nm} \end{pmatrix};$$

- матрица запросов размером $m \times n$, определяющая потребности процессов в ресурсах каждого типа:

$$R = \begin{pmatrix} R_{11} & R_{12} & \dots & R_{1m} \\ R_{21} & R_{22} & \dots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & \dots & R_{nm} \end{pmatrix};$$

- при этом количество свободных и распределенных ресурсов не должно превышать количество доступных ресурсов:

$$E_j = \sum_{i=1}^n C_{ij} + A_j.$$

Алгоритм состоит из следующих шагов:

1. Поиск непомеченного процесса P_i , для которого i -я строка матрицы R меньше или равна A .
2. Если такой процесс найден, прибавляем к A i -ую строку матрицы C . Установка метки на i -ый процесс, переход к шагу 1.
3. Если такого процесса нет, алгоритм прекращает работу.

По окончании работы алгоритма все непомеченные процессы, считаются участвующими во взаимоблокировке.

Рассмотрим пример работы алгоритма. Допустим имеются следующие процессы и ресурсы:

- вектор существующих ресурсов $E = (4, 2, 3, 1)$;
- вектор доступных ресурсов $A = (2, 1, 0, 0)$;
- матрица текущего распределения:

$$C = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{pmatrix};$$

- матрица запросов:

$$R = \begin{pmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{pmatrix}.$$

Тогда представленный алгоритм будет выполняться следующим образом:

1. Третий процесс может получить желаемые ресурсы, т.к. $R_3 = A$.

2. Третий процесс освобождает ресурсы $A = (2, 2, 2, 0)$. Процесс маркируется как выполненный.
3. Выполняется процесс 2. По окончании $A = (4, 2, 2, 1)$.
4. Выполняется первый процесс. Тупиков не обнаружено.

Если изменить запросы для второго процесса со значений $R_2 = (1, 0, 1, 0)$ на $R_2 = (1, 0, 3, 0)$, тогда матрица запросов будет иметь вид:

$$R = \begin{pmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 3 & 0 \\ 2 & 1 & 0 & 0 \end{pmatrix},$$

а два процесса окажутся в тупике.

Когда в системе обнаружена взаимоблокировка необходимо восстановить работоспособность системы с помощью следующих методов:

1. Восстановление за счет приоритетного владения ресурсом. При возникновении взаимоблокировки можно отобрать ресурсы у других процессов, выбор таких процессов должен быть сделан пользователем.
2. Восстановление путем отката. Состояние процессов записывается в контрольные точки. При возникновении взаимоблокировки существует возможность выполнить откат состояния процесса на некоторое время назад.
3. Восстановление путем уничтожения процессов. При уничтожении процессов освобождаются ресурсы, занятые ими. Выбор процессов должен быть сделан пользователем.

Динамическое уклонение за счет тщательного распределения ресурсов

При динамическом распределении ресурсов операционная система должна определять текущее состояние: безопасное или небезопасное.

В безопасном состоянии взаимоблокировка гарантированно не произойдет. В небезопасном состоянии существует возможность возникновения взаимоблокировки, но взаимоблокировка может и не произойти.

Для определения состояния может быть применен алгоритм банкира:

1. Банкиру поступает запрос от клиента на получение кредита.

2. Банкир проверяет, приводит ли этот запрос к небезопасному состоянию. В безопасном состоянии система может гарантировать, что все процессы закончат свою работу.
3. В зависимости от состояния банкир выдает или отказывает в выдаче кредита.

Для определения состояния системы может быть использован следующий алгоритм. Введем некоторые обозначения:

- вектор существующих ресурсов $E = (E_1, E_2, \dots, E_m)$;
- вектор доступных ресурсов $A = (A_1, A_2, \dots, A_m)$;
- вектор занятых ресурсов $P = (P_1, P_2, \dots, P_m)$, каждый элемент которого рассчитывается как разность между существующими и доступными ресурсами $P_i = E_i - A_i$;
- матрица текущего распределения:

$$C = \begin{pmatrix} C_{11} & C_{12} & \dots & C_{1m} \\ C_{21} & C_{22} & \dots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \dots & C_{nm} \end{pmatrix};$$

- матрица запросов:

$$R = \begin{pmatrix} R_{11} & R_{12} & \dots & R_{1m} \\ R_{21} & R_{22} & \dots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & \dots & R_{nm} \end{pmatrix};$$

- при этом количество свободных и распределенных ресурсов не должно превышать количество доступных ресурсов.

Алгоритм состоит из следующих шагов:

1. Поиск процесса в матрице R , потребности которого меньше или равны вектору A . Если искомая строка не существует, то система окажется в состоянии взаимной блокировки. Прекращение работы алгоритма.
2. Предполагается, что процесс завершился и освободил занимаемые им ресурсы. Процесс помечается как завершённый, освободившиеся ресурсы добавляются в вектор A .
3. Повторение шагов 1 и 2 пока все процессы не будут помечены как завершённые (безопасное состояние), либо не останется процессов, чьи запросы могут быть удовлетворены (небезопасное состояние).

Рассмотрим пример работы алгоритма. Допустим имеются следующие процессы и ресурсы:

- вектор существующих ресурсов $E = (6, 3, 4, 2)$;
- вектор доступных ресурсов $A = (1, 0, 2, 0)$;
- вектор занятых ресурсов $P(5, 3, 2, 2)$;
- матрица текущего распределения:

$$C = \begin{pmatrix} 3 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix};$$

- матрица запросов:

$$R = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 2 \\ 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 2 & 1 & 1 & 0 \end{pmatrix}.$$

Изначально текущее состояние системы является безопасным. Предположим, что процесс 2 сделал запрос на получение одного ресурса второго типа. После выполнения алгоритма состояние системы также определяется как безопасное. Следовательно, запрос может быть удовлетворен.

Предотвращение за счет подавления одного из условий возникновения

Также возникновение тупиковых ситуаций можно предотвратить за счет структурного подавления одного из четырех условий возникновения взаимоблокировок:

1. Предотвращение условия взаимного исключения. Минимизация количества процессов борющихся за ресурсы. Например, с помощью очереди печати, когда только демон/сервис принтера работает с ним.
2. Предотвращение условия удержания и ожидания. Один из способов – запрос процессом всех необходимых ресурсов до начала работы. В случае недоступности хотя бы одного ресурса процессу ничего не предоставляется.

3. Предотвращение условия отсутствия принудительной выгрузки ресурса.
4. Предотвращение условия циклического ожидания:
 - процесс должен освободить занятый ресурс, прежде чем занять новый;
 - нумерация всех ресурсов. Процесс может запрашивать ресурсы только в порядке возрастания их номеров.

6.3 Контрольные вопросы

1. Что такое состязательная ситуация?
2. Какие механизмы решения проблемы возникновения состязательных ситуаций существуют?
3. Что такое тупиковая ситуация?
4. Какие виды ресурсов существуют?
5. Перечислите условия возникновения взаимоблокировки.
6. Какие методы борьбы с взаимоблокировками существуют?
7. Каким образом можно восстановить состояние операционной системы после возникновения взаимоблокировки?
8. В чем разница между безопасным и небезопасным состояниями?
9. Какие методы подавления условий возникновения взаимоблокировок существуют?

ЗАКЛЮЧЕНИЕ

Данное учебное пособие направлено на формирование у студентов профессиональных компетенций, связанных с использованием теоретических знаний о концепциях, принципах, алгоритмах, структурах и моделях, положенных в основу современных операционных систем. Особое внимание уделяется изучению абстракций операционных систем: процессы, потоки, виртуальная память, файл, ввод/вывод и т. д.

Задачами данного пособия являются:

- изучение принципов, заложенных в основу современных операционных систем;
- формирование навыков работы с различными видами абстракций;
- рассмотрение и изучение применения различных подходов к построению операционных систем при решении других задач;
- приобретение теоретических знаний и практических навыков разработки программных систем, оперирующих абстракциями.

Получение студентами теоретических знаний и практических навыков применения и построения абстракций необходимо для разработки и поддержания жизнеспособности современных программных систем.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Таненбаум Э. С., Херберт Б. Современные операционные системы. 4-е изд. — Издательский дом «Питер», 2015.

Учебное издание

ФИЛИППОВ Алексей Александрович

Операционные системы

Учебное пособие

Редактор ??. ???????

Оригинал-макет изготовлен в системе $\text{\LaTeX}2_{\epsilon}$.

ЛР № 020640 от 22.10.97.

Подписано в печать ??.??.????.

Формат 60×84/16. Усл. печ. л. 6,74.

Тираж ??? экз. Заказ ?.

Ульяновский государственный технический университет,
432027, Ульяновск, Сев. Венец, 32.

ИПК «Венец», УлГТУ, 432027, Ульяновск, Сев. Венец, 32.